

队伍编号	MCB2401427
赛道	(B)

基于数据挖掘的电商品类货量预测与分仓规划研究

摘 要

在现代商业环境中，仓库的运营往往被看作是日常运作中不可或缺的一部分。本文对品仓分类问题进行了深入研究，从问题深度来说，将问题分解为：精准预测货量；根据货量预测结果进行“一品一仓分仓方案”的规划；对于新增维度下的“一品多仓分仓方案”的规划。

针对问题 1：对于如何精准预测货量。首先进行**数据清洗**，利用 Pandas 中 isnull 方法查询缺失值，利用四分位距检测异常值，并采用多项插值法替换出现的异常值。然后基于附件 1 和附件 2 的相关数据，以 80%的数据为训练集，以 20%的数据为测试集依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络，得到五个模型的训练集真实值、测试集真实值、各模型预测值、各模型 MAPE 值以及各模型评估参数对比。最后选择**LSTM 神经网络**预测月库存量，其中精确度达 0.836，MAPE 值为 0.011；**BP 神经网络**预测日销量，其中精确度达 0.814，MAPE 值为 0.013，预测结果见附件 1 中的 result 表格。

针对问题 2：根据货量预测结果进行“一品一仓分仓方案”的规划。首先需要对附件 3 中的数据进行预处理，将问题 1 所得每天销售量的数据集按每月进行**归类**。可将问题 2 进行简化，即将“一品一仓”问题抽象成**0-1 规划**问题。然后根据题目的要求综合考虑多个业务目标并确定约束条件，从而建立“一品一仓”方案模型。最后基于附件 3 的相关数据，采用**分枝定界法**利用 Lingo 软件以及**SCIP 求解器**进行求解，直至容差界限小于 0.001，此时用时 86 秒，迭代次数 3941 次，说明了该算法在求解过程中的**高效性**和**稳定性**，预测结果见附件 1 中的 result 表格。

针对问题 3：对于新增维度下的“一品多仓分仓方案”的规划。首先基于问题一中所得“月库存预测结果”和“日销量预测结果”两个工作表的数据，读取题中所给的附件 3、附件 4、附件 5 相关数据，**构建品类关联度矩阵**，并对各指标设置**权重系数**。接着建立目标函数，最大化仓库中品类的关联度，通过**添加松弛变量**并在目标函数中给予惩罚权重，确定约束条件。然后分别建立多目标模拟退火算法和多目标粒子群优化算法，通过对比两个算法的收敛图效果，最后选取**多目标粒子群优化算法**，预测结果见附件 1 中的 result 表格中。

关键词：电商品类；分仓规划；LSTM 神经网络；BP 神经网络；分枝界定法；多目标粒子群优化算法

目 录

1 问题重述	1
1.1 问题背景	1
1.2 问题提出	2
1.3 研究综述	2
2 问题分析	3
2.1 问题 1 的分析	3
2.2 问题 2 的分析	4
2.3 问题 3 的分析	4
3 模型假设	5
4 符号说明	6
5 问题 1 的模型建立与求解	6
5.1 数据预处理	6
5.1.1 缺失值的识别与处理	7
5.1.2 异常值的识别与处理	7
5.2 子问题 a) 的建模与求解	8
5.2.1 子问题 a) 模型的建立	8
5.2.2 子问题 a) 模型的求解	13
5.3 子问题 b) 的建模与求解	18
5.3.1 子问题 b) 模型的建立	18
5.3.2 子问题 b) 模型的求解	19
6 问题 2 的模型建立与求解	23
6.1 数据预处理	23
6.2 问题 2 的建模与求解	23
6.2.1 问题 2 模型的建立	23
6.2.2 问题 2 模型的求解	25
7 问题 3 的模型建立与求解	27
7.1 数据预处理	27
7.2 问题 3 的建模与求解	28
7.2.1 问题 3 模型的建立	28
7.2.2 问题 3 模型的求解	30
8 模型的评价和推广	34
8.1 模型的评价	34
8.1.1 模型的优点	34
8.1.2 模型的不足	35
8.2 模型的推广	35
8.2.1 科学性	35
8.2.2 拓展性	35
参考文献	35
附录	37
1. 支撑文件列表	37
问题 1-3 计算结果, 见附件 1.	37

问题 1 支撑材料，见附件 2.....	37
问题 2 支撑材料，见附件 3.....	37
2.附录内容.....	37
附录一:问题 1 的缺失值处理.....	37
附录二:问题 1 的异常值处理.....	38
附录三:问题 1 的库存量预测.....	40
附录四:问题 1 的销量预测.....	48
附录五:问题 2 的“一品一仓分仓方案”.....	56
附录六:问题 3 的“一品多仓分仓方案”.....	59

表 录

表 1 附件 1 与附件 2 缺失值识别结果	7
表 2 附件 1 异常值处理结果	8
表 3 附件 2 异常值处理结果	8
表 4 附件 1 五种模型评估参数对比	15
表 5 月库存量预测结果	18
表 6 附件 2 五种模型评估参数对比	21
表 7 模型评估结果	23
表 8 日销量预测结果	23
表 9 分枝定界法具体步骤	26
表 10 “一品一仓”分仓方案	27
表 11 “一品多仓”分仓方案	34

图 录

图 1 商品分仓示意图.....	1
图 2 关键词共现图谱.....	3
图 3 问题 1 分析流程图.....	4
图 4 问题 2 分析流程图.....	4
图 5 “一品多仓”业务目标.....	5
图 6 问题 3 分析流程图.....	5
图 7 附件 1 与附件 2 异常值识别结果.....	8
图 8 随机森林的基本思想.....	9
图 9 BP 神经网络结构图.....	11
图 10 BP 神经网络流程图.....	12
图 11 LSTM 神经网络结构图.....	13
图 12 五种模型的训练集、测试集真实值以及各模型库存量预测值.....	14
图 13 附件 1 五种模型的 MAPE 比较.....	15
图 14 附件 1 数据量维度.....	16
图 15 附件 1 损失函数结果.....	16
图 16 附件 1 SSE 随簇数变化曲线.....	17
图 17 附件 1 数据集聚类结果.....	17
图 18 附件 1 聚类处理后损失函数结果.....	18
图 19 五种模型的训练集、测试集真实值以及各模型销售量预测值.....	19
图 20 附件 2 五种模型的 MAPE 比较.....	20
图 21 附件 2 差分结果.....	21
图 22 Z-score 标准化结果.....	22
图 23 模型迭代次数.....	22
图 24 商品分仓问题示例.....	24
图 25 分枝定界法算法流程图.....	26
图 26 上下界收敛曲线.....	27
图 27 品类关联度矩阵.....	28
图 28 模拟退火算法示意图.....	30
图 29 模拟退火算法流程图.....	31
图 30 模拟退火算法收敛图.....	31
图 31 粒子群算法流程图.....	32
图 32 粒子群算法收敛图趋势.....	33

1 问题重述

1.1 问题背景

进入 21 世纪，随着互联网技术的快速发展和普及，中国的电子商务平台如雨后春笋般崭露头角。历经二十余载的迅猛发展，电子商务产业已成功构建起一个规模宏大、结构健全的产业体系。如今，电子商务平台不仅颠覆了传统的商业运作模式，更在潜移默化中重塑着人们的消费观念与生活方式。与此同时，随着民众生活质量的稳步提升，消费观念亦在悄然演进。消费者对于商品的需求，已从往昔单纯注重经济实用性，转向追求更为个性化与多元化的体验。他们不再仅仅满足于商品的基本功能，而是更加注重商品所蕴含的文化底蕴与所展现的品牌形象价值。这一消费观念的转变，促使电商企业必须更加敏锐地捕捉市场动态，提供丰富多样、高度个性化的商品与服务，以满足消费者的多元化需求。

然而，随着电商业务的蓬勃发展，库存管理问题逐渐浮出水面。品类分仓规划决定各商品存放在哪些仓库问题，合理的品类分仓规划对提升每个仓的管理效率、降低总体仓储成本至关重要，这给电商公司的库存管理带来了极大的挑战。库存管理的优化不仅关乎到公司的成本控制，更直接影响到公司的市场竞争力和长期发展。库存作为电商业务的核心环节之一，其管理策略直接关系到企业的运营效率和经济效益。然而，在现实中，由于商品种类繁多，且每种商品的数量可能也很大，因此需要将这些商品合理地分配到不同的仓库中进行存储，如图 1 所示。首先，合理的品类分仓规划可以提升每个仓库的管理效率。其次，合理的品类分仓规划还可以降低总体仓储成本。许多电商企业往往面临库存计划不精准的问题，导致短缺与积压并存。消费者需要的商品缺货，而不需要的商品却积压过多。这种高缺货率和高呆滞库存比例不仅直接导致了业务收入的损失，更使得库存商品的价值逐渐贬值。为了缓解库存压力，企业不得不采取削价清仓的方式，这无疑进一步加剧了成本负担，给企业带来了沉重的经济压力。



图 1 商品分仓示意图

要解决上述问题，需求预测成为关键。在需求预测方面，电商往往存在过度依赖销售数据作为预测的唯一依据，忽视了其他可能影响需求的因素，如库存量、仓容及产能等。这种单一的预测方法往往导致预测结果的不准确和偏差。因此，本文选择基于需求预测的电商公司库存管理优化作为研究，旨在通过深入研究需求预测的方法和技术，探索出更加科学、准确的库存管理方法。通过优化库存管理，确保库存大概率满足消费者需求、避免

断供的前提下，提高仓容利用率和产能利用率，从而达到降低库存成本的目的。这不仅有助于提升电商公司的运营效率和竞争力，也为整个电商产业的库存管理提供了有益的借鉴和参考。

1.2 问题提出

为了研究商品分仓问题,我们团队对附件中所提供的数据和相关材料进行了全面处理和深入分析,我们的研究重点围绕以下三个问题展开:

➤ 问题 1: 建立“货量预测”模型

根据附件 1 和附 2 各品类的历史库存量、销量来预测未来 3 个月每个品类的月库存量和每日销量。库存量和销量的准确预测可以支持后续的分仓决策，提前安排仓储资源，减少冗余和缺货风险。并将月库存预测结果和日销量预测结果填入在 result 表格。

➤ 问题 2: 设计“一品一仓”分仓方案

假设每个品类只能放一个仓库中，基于问题 1 的预测结果，设计一个最优的分仓方案。根据附件 3 提供不同仓库相关信息，需要在仓库容量、仓库产量、仓租成本之间找到平衡，以实现最低仓租成本和资源的高效利用。并将完整“一品一仓”分仓结果填入在 result 表格。

➤ 问题 3: 设计“一品多仓”分仓方案

依据附件 4 和附 5 五各品类之间的关联度以及相关信息，允许一个品类最多可以分布在 3 个仓库中，并尽量将同伴型、同高级品类放在同一个仓库中。通过优化品类关联度，减少订单分拆，降低履约成本。此问题是多目标优化问题，需在品类关联度、仓容和产能利用率及仓租成本之间取得平衡。并将完整“一品多仓”分仓结果填入在 result 表格。

1.3 研究综述

鉴于仓库管理问题研究在物流效率、成本控制等方面起关键性作用，对于这一领域的分析和预测研究一直没有间断。尤其是最近几年，大数据技术的崛起为仓库管理问题研究提供了新的机遇，通过收集和分析大规模的销量数据和库存量数据，研究者可以更加全面地理解仓库运行机制，控制风险因素，相关数学模型变得更加的科学新颖，对智能出货的结果也变得更加的精确。因此，本次建模会借鉴已有的相关研究成果，参考已有研究成果的建模思路、技术路线和结论，在文献综述的基础上进一步探究出“货量预测”模型的建立问题、“一品一仓”分仓方案以及“一品多仓”分仓方案问题。

中国知网是中国综合性学术文献资源平台，包括学术期刊论文、学位论文、会议论文、报纸、图书、法规、统计数据等各类信息资源。本次文献资料来源于中国知网数据库，在高级检索模式下，以主题检索词为“分仓规划”并含“数学模型”进行检索，检索方式设置为精确，最终得到文献 38 篇，由文献总量可知，我国对利用数学模型研究分仓规划问题中还在起步阶段，需要进一步拓展研究范围，提高研究层次。本次研究综述所采用的工具是陈超美教授团队开发的 Citespace 软件，将以上文献导入到 Citespace 中并进行可视化分析^[1]。

关键词能鲜明且直观地反映文献所要论述的主题，而高频关键词所表征的研究主题则能揭示相关领域的研究热点。将利用数学模型研究分仓规划中的相关文献关键词作为分析对象导入 CiteSpace，得到包含包含 104 个节点、143 条连线、网络整体密度为 0.0267 的关键词共现网络图谱，如图 2 所示。由图可知，在数学模型上主要以预测模型为主，包含 BP 神经网络、多元线性回归、Logistic 分类、XGBoost 算法、SVM 算法等。

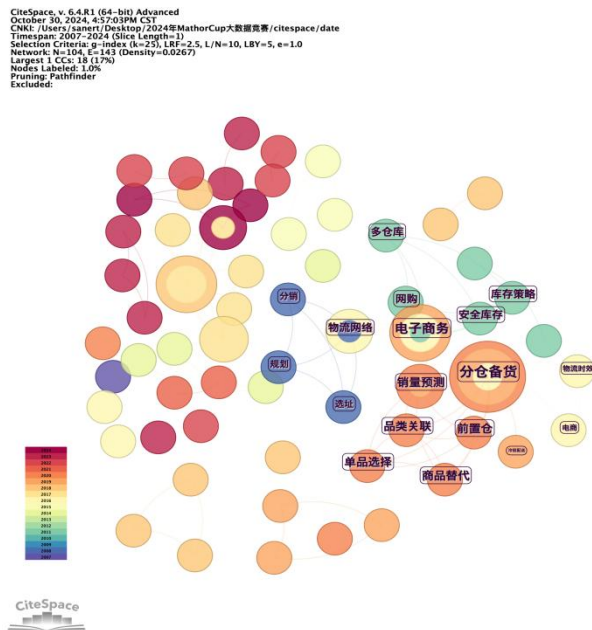


图 2 关键词共现图谱

在已有研究成果中，李长春对交易数据进行特征分析，通过对灰色预测模型、神经网络模型、多元回归模型、随机森林模型等模型预测性能的比较，最终选取随机森林模型的预测结果，建立报童模型求解最优的分仓库存储规划^[2]。

王之威以优化分仓物流网络布局建立最小运输仓储费用的多产品三阶段物流网络模型，并进行实证研究，采用 Lingo 求解取得较好的优化结果^[3]。

刘顺等人以最小化被拆分订单的总量为目标，基于历史订单数据构建超图作谱图分解，获取各商品的嵌入表示，利用上、下限簇数限制的约束聚类算法生成商品分仓的结果^[4]。

利用 Citespace 的关键词突现功能，以了解目前预测研究的热点、前沿以及发展趋势。由图可知，商品分仓规划问题仍然是目前的研究热点。因此，本次的建模问题契合目前研究的发展趋势，使得问题本身具有更加重要的现实意义。

2 问题分析

2.1 问题 1 的分析

针对问题 1，需要首先对附件 1 和附件 2 中的数据进行预处理，利用 Pandas 中 isnull 方法查询缺失值，利用四分位距检测异常值，并采用多项插值法替换出现的异常值。基于附件 1 和附件 2 的相关数据，以 80% 的数据为训练集，以 20% 的数据为测试集依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络，得到五个模型的训练集真实值、测试集真实值、各模型预测值、各模型 MAPE 值以及各模型评估参数对比，从而选出最合适的模型分别预测问题 1 的子问题 a) 中的月库存量和问题 1 的子问题 b) 中的日销量，问题 1 流程见图 3。

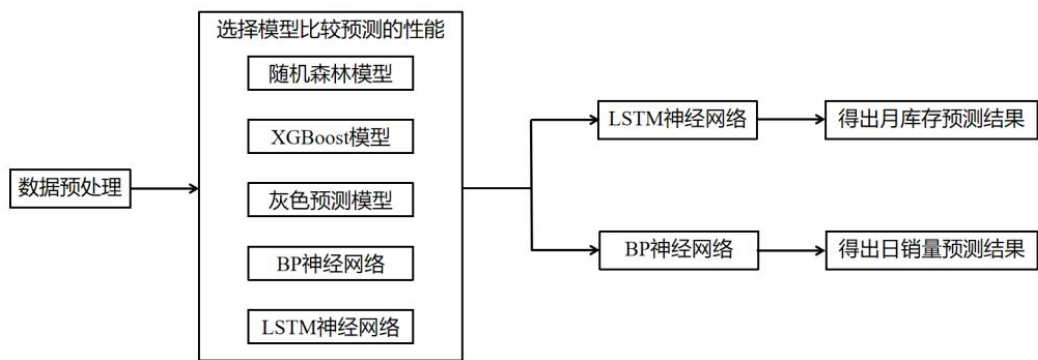


图 3 问题 1 分析流程图

2.2 问题 2 的分析

针对问题 2，需要首先对附件 3 中的数据进行预处理，将问题 1 所得每天销售量的数据集按每月进行归类。可将问题 2 进行简化，即将“一品一仓”问题抽象成 0-1 规划问题。目标函数可围绕每个商品都有位置存放，并且仓库数量及时长最少的情况下，总成本最少进行设立。由于需要综合考虑多个业务目标，约束条件的确定至关重要，从而建立“一品一仓”方案模型。基于附件 3 的相关数据，采用分枝定界法利用 Lingo 软件进行求解，从而得出“一品一仓”最优解，问题 2 流程见图 4。

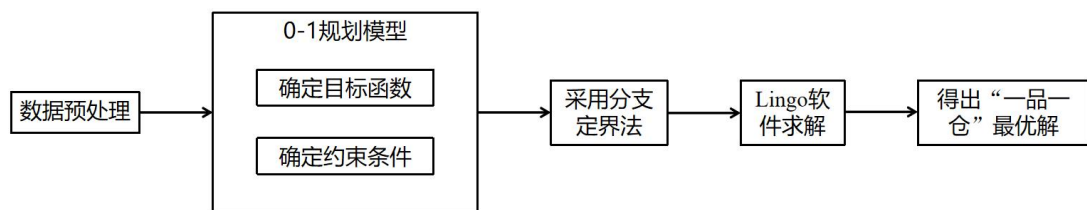


图 4 问题 2 分析流程图

2.3 问题 3 的分析

针对问题 3，在问题 2 的基础上放开了一品一仓的限制，允许一个品类最多分配给三个仓库，同时引入了件型和高级品类的标签约束，构成了“一品多仓”业务目标，如图 5 所示，这使得问题变得更加复杂但也更贴近实际业务场景，新增的约束条件要求同伴型、同高级品类的商品尽量集中存放，这实际上增加了一个聚类的维度，需要在分仓规划时同时考虑品类的相似性和关联性，而且品类在不同仓库之间的库存量和出库量比例需要保持一致，这给优化带来了新的挑战。

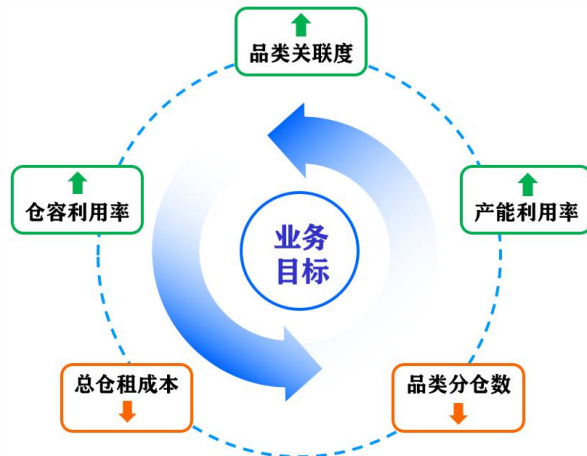


图5 “一品多仓”业务目标

从解决方案的角度来看，可以考虑采用多目标优化的策略，可以先基于件型和高级品类的标签对商品进行预聚类，将相似的商品组合在一起，然后在此基础上进行仓库分配优化，这种方法可以有效降低问题的复杂度，使解决方案更具可操作性，同时需要设计合理的评估机制，用于比较不同方案下各个业务指标的表现，找到多个目标之间的最佳平衡点，问题3流程见图6。

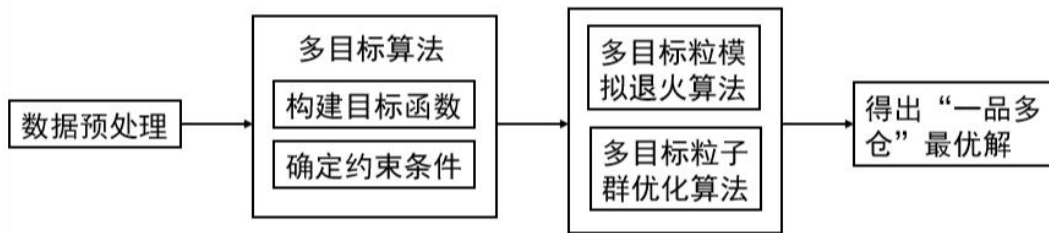


图6 问题3分析流程图

3 模型假设

假设一：假设在电商供应链系统中，所有品类的需求是独立于其他品类的，即每个品类的需求和其他品类的需求之间不存在直接的因果关系，这使得我们可以对每个品类独立进行需求预测，同时在预测过程中考虑品类之间的相关性来提高预测准确性。

假设二：假设每个仓库的容量上限和产能上限在规划周期内是固定的，且这些限制在规划期间内不会发生变化。这保证了仓库的基础设施和作业能力在短期内是稳定的，不会因为季节变化或其他因素而发生显著变化，这为分仓规划提供了稳定的约束条件。

假设三：假设品类之间的关联度是基于历史销售数据或其他可靠方法计算得出的，能够反映品类之间的销售相关性。且在未来的规划期内保持相对稳定，这种假设使得我们可以利用历史数据中的关联关系来指导未来的分仓决策，从而提高订单履约效率。

假设四：训练集数据能够充分代表整个时间序列的特征和趋势，测试集数据能够反映时间序列的最新趋势和变化。

假设五：数据预处理步骤（如缺失值处理、异常值检测和处理、数据标准化等）是有效的，并且不会对模型性能产生负面影响。

4 符号说明

序号	符号	含义	序号	符号	含义
1	$\hat{y}_i$	第 i 个样本的预测值	20	y_t	实际数据
2	L	损失函数	21	e_t	预测误差
3	Ω	正则项	22	$\hat{y}_t$	预测值
4	$Obj^{(t)}$	目标函数	23	$\bar{y}_t$	平均值
5	$\hat{a}$	参数列	24	x_{ij}	第 i 个产品对第 j 个仓库
6	x_n	输入变量	25	P	总成本
7	w_{ij}	层间的连接权值	26	p_j	各仓库使用单价
8	F_t	遗忘门	27	t_j	商品放置时间的长短
9	I_t	输入门	28	B_i	第 i 个产品每个月的销量
10	O_t	输出门	29	h_j	每个产品的仓库储存的状态
11	σ	<i>Sigmoid</i> 函数	30	D_j	第 j 个仓库的仓容
12	W	权重	31	C_j	第 j 个仓库的产能
13	H_{t-1}	上一时刻单元输出	32	f_1	品类关联度
14	b	偏置项	33	W_k	仓库 k 中分配的品类集合
15	N_t	更新信息	34	$ W_k $	仓库 k 中品类的数量
16	$Tanh$	状态激活函数	35	f_2	产能利用率
17	C_t	更新的单元状态	36	f_3	仓能利用率
18	μ	均值	37	f_4	总仓租成本
19	δ	标准差	38	f_5	品类分仓数

5 问题 1 的模型建立与求解

5.1 数据预处理

在进行仓储网络库存量与销量的预测时，数据预处理是一个至关重要的步骤。数据的质量，直接决定了模型的预测和泛化能力的好坏。它涉及很多因素，包括：准确性、完整性、一致性、时效性、可信性和解释性。而在真实数据中，我们拿到的数据可能包含了大量的缺失值，可能包含大量的噪音，也可能因为人工录入错误导致有异常点存在，非常不利于算法模型的训练。数据清洗的结果是对各种脏数据进行对应方式的处理，得到标准的、干净的、连续的数据，提供给数据统计、数据挖掘等使用。具体来说，历史数据中可能存在缺失值，如某些仓库在某些日期的库存量或销量未记录。这些缺失值会影响预测模型的准确性和稳定性，因此需要通过插值、填充平均值或利用其他统计方法进行处理。同时，

数据中可能包含异常值（如销量大于库存量时就不符合逻辑），这些异常值可能是由于记录错误或特殊事件导致的。它们会扭曲数据的真实分布，影响预测模型的性能，因此需要识别并适当处理这些异常值。当缺失数据和异常数据的占比非常小时，可以将忽略缺失数据并将异常数据视为正常数据进行分析和处理；当缺失数据和异常数据的占比无法忽略时，需要采用必要的数据处理方法以维持数据的完整性和时序性。

5.1.1 缺失值的识别与处理

（1）附件 1 与附件 2 缺失值的识别

缺失值即空值。数据集不含缺失值的变量被称为完全变量，含有缺失值的变量被称为不完全变量。在 Pandas 中查询缺失值，最常用的就是 `isnull` 方法，返回 `True` 表示此处为缺失值。利用 python，输入附件 1 数据，输出结果如表 1 所示。

表 1 附件 1 与附件 2 缺失值识别结果

附件	品类别空值个数（单位：个）	月份列缺失值个数（单位：个）	库存量缺失值个数（单位：个）
附件 1	0	0	0
附件 2	0	0	0

（2）附件 1 与附件 2 缺失值的处理

由表 1 可知，附件 1 与附件 2 无缺失值，因此无需做缺失值处理。

5.1.2 异常值的识别与处理

（1）附件 1 与附件 2 异常值的识别

异常值可能是离群点，即与整体数据情况偏离很大的数据点（常见的四分位距），也可能是超过某个不合理范围的数据点，在分析中应该首先排除掉异常值，异常值检测逻辑是对变量的数据集按照设置的或值进行判定，筛选出落在异常值检测范围内的数据，并对异常值进行置空或者是填补为其它有效值。本研究使用四分位距检测异常值，四分位距是指第一四分位数（ Q_1 ，即 25%分位数）与第三四分位数（ Q_3 ，即 75%分位数）之间的差值。它衡量了数据中间 50%范围的离散程度，相较于标准差或方差，四分位距对于极端值（即异常值）的敏感度较低，因此能够更稳健地反映数据的中心分布情况。通过将数据集按大小排序后分为四等分，计算第一四分位数（ Q_1 ）和第三四分位数（ Q_3 ）之间的差值，我们可以得到四分位距。计算公式为：

$$IQR = Q_3 - Q_1 \quad (5-1)$$

四分位距的值越小，说明中间的数据越集中；值越大，则说明中间的数据越分散。这种特性使得四分位距成为衡量数据离散程度的一个稳健指标，特别适用于存在极端值或偏态分布的数据集。它假设如果数据点比 25%四分位数低 1.5 倍的四分位距或比 75%四分位数高 1.5 倍的四分位距，则被视为异常值。这种方法不需要对数据进行正态分布的假设，因此适用于各种类型的数据集。利用 python，输入附件 1 与附件 2 的数据，输出结果如图 7 所示。

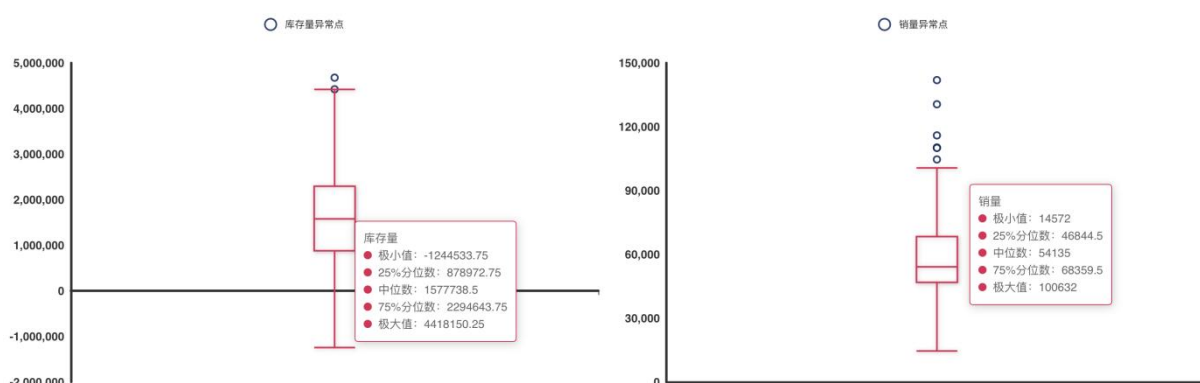


图 7 附件 1 与附件 2 异常值识别结果

(2) 附件 1 与附件 2 异常值的处理

由图 7 可知，附件 1 与附件 2 均存在异常值，因此需对其进行处理。研究采用多项插值法替换出现的异常值，这样能够保持数据的连续性和趋势，处理结果如表 2 和表 3 所示。处理异常值后的数据集见附件 2-1 和附件 2-2。

表 2 附件 1 异常值处理结果

附件 1 产品	时间	库存量	插值结果
category84	2023/1/1	4421974	1558725
category225	2023/6/1	4676058	1562173

表 3 附件 2 异常值处理结果

附件 2 产品	时间	销量	插值结果
category21	2022/8/31	130556	67458
category21	2022/8/29	110020	57482
category21	2022/9/30	104616	55892
category84	2023/6/1	115928	58740
category84	2023/6/18	141914	68984

5.2 子问题 a) 的建模与求解

5.2.1 子问题 a) 模型的建立

(1) 随机森林模型

随机森林是一种基于决策树的集成学习算法，可以用于回归和分类问题，其基本思想见图 8。随机森林算法可以通过计算变量特征在森林中多次出现的情况来评估变量的特征重要性^[5]。在训练过程中，随机森林会对每个决策树使用基于随机样本和随机特征的方式进行训练。因此，每个决策树的特征重要性可以通过计算每个特征在每个决策树中的平均不纯度改善量来得到。然后，可以将每个特征在所有决策树中的平均不纯度改善量进行累加，得到变量特征重要性的总体评估。

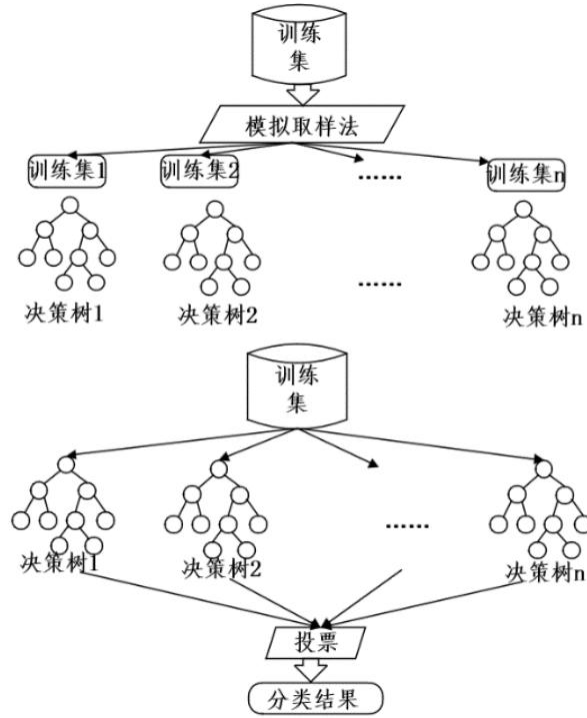


图 8 随机森林的基本思想

(2) XGBoost 模型

XGBoost 是一种有效的机器学习方法，它是一种以梯度为基础的集成学习方法。XGBoost 在包括 Kaggle、Tianchi 在内的众多机器学习比赛中都取得了优异的成绩。对于包含 n 条 m 维的数据集，XGBoost 模型可表示为：

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in F (i = 1, 2, \dots, n) \quad (5-2)$$

其中，

$$F = \{f(x) = w_{q(x)}\} (q: R^m \rightarrow 1, 2, \dots, T, w \in R^T) \quad (5-3)$$

是一组 CART 决策树，它是将样本映射到树叶结点上的的一种树形结构，表示树叶结点的数目，表示叶结点的真实得分。在构造 XGBoost 模型时，应按照优化目标函数的最小化原理，求出最佳的参数。XGBoost 模型中的目标函数可以分成两类，一类是误差函数，一类是模型复杂性。

目标函数可写为：

$$Obj = L + \Omega \quad (5-4)$$

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5-5)$$

$$\Omega = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (5-6)$$

在利用训练样本对模型进行最优学习的时候，要保持原模型的不变性，向模型中添加新的函数，使得目标函数最小化。

$$\begin{aligned}
\hat{y}_i^{(0)} &= 0 \\
\hat{y}_i^{(1)} &= \hat{y}_i^{(0)} + f_1(x_i) \\
\hat{y}_i^{(2)} &= \hat{y}_i^{(1)} + f_2(x_i) \\
&\dots\dots \\
\hat{y}_i^{(t)} &= \hat{y}_i^{(t-1)} + f_t(x_i)
\end{aligned} \tag{5-7}$$

此时目标函数表示为：

$$Obj^{(t)} = \sum_{i=1}^n (y_i - (\hat{y}_i^{(t-1)} + f_i(x_i)))^2 + \Omega \tag{5-8}$$

对目标函数进行了二阶泰勒展开，得到近似目标函数：

$$Obj^{(t)} \approx \sum_{i=1}^n [(y_i - \hat{y}_i^{(t-1)})^2 + 2(y_i - \hat{y}_i^{(t-1)})f_t(x_i) - h_i f_t^2(x_i)] + \Omega \tag{5-9}$$

当去掉常数项后可知：目标函数仅仅与误差函数的一阶和二阶导数相关。此时，目标函数表示为：

$$\begin{aligned}
Obj^{(t)} &\approx \sum_{i=1}^n [g_i w_{q(x_i)} + \frac{1}{2} h_i w_{q(x_i)}^2] + \gamma T + \frac{1}{2} \sum_{j=1}^T w_j^2 \\
&= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T
\end{aligned} \tag{5-10}$$

若树的结构部分 q 已知，可使用目标函数寻找最优 w_j ，并得到最优目标函数值。其本质可归为二次函数的最小值求解问题。解得：

$$w_j^* = \frac{-\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{5-11}$$

$$Obj = -\frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \tag{5-12}$$

Obj 是可作为评价模型的打分函数， Obj 值越小则模型效果越好。通过递归调用上述树的建立方法，可得到大量回归树结构，并使用 Obj 搜索最优的树结构，将其放入已有模型中，从而建立最优的 XGBoost 模型。

(3) 灰色预测模型——GM (1, N) 预测模型

GM (1, N) 模型是一种基于灰色系统理论的预测模型，是灰色预测的一种扩展，它用于处理含有 1 阶方程和 N 个变量的系统，适用于多变量的情况，特别适用于那些受多种因素影响的预测问题，专门用于处理不完全信息系统的数据预测问题。这种模型通过累加生成、最小二乘法、时间响应函数等手段，挖掘数据间的内在规律，从而实现对未来数据的预测^[6]。例如，当需要预测一个仓库的库存量时，该库存量可能不仅与时间有关，还可能受到仓库的租用成本、仓容利用率、品类分仓数等多个因素的影响。在这种情况下，GM(1,N)模型就可以用来从有限的、不完全的数据中找到这些因素与仓库的仓库量之间的关系。

设变量 $x^{(0)}$ 的原始数据序列如下：

$$x^{(0)} = \{x^{(0)}(1), x^{(0)}(2), \dots, x^{(0)}(n)\} \quad (5-13)$$

对 $x^{(0)}$ 进行累加，生成一阶累加生成序列 $x^{(1)}$ ：

$$x^{(1)} = \{x^{(1)}(1), x^{(1)}(2), \dots, x^{(1)}(n)\} \quad (5-14)$$

其中， $x^{(1)}(k) = \sum_{i=1}^k x^{(0)}(i)$ ， $(k = 1, 2, 3 \dots n)$ ，对 $x^{(1)}$ 构建白化微分方程模型：

$$\frac{dx_1^{(1)}}{dt} + ax_1^{(1)} = b_1x_2^{(1)} + b_2x_3^{(1)} + \dots + b_{N-1}x_N^{(1)} \quad (5-15)$$

这个一阶 N 个变量的微分方程模型记为 GM(1,N)，上述方程的参数列为 $\hat{a} = [a, b_1, b_2, \dots, b_{N-1}]$ 按最小二乘法可求出 $\hat{a}$ ，其算式为 $\hat{a} = (A^T A)^{-1} A^T Y$ 。

其中， A 与 Y 分别为：

$$A = \begin{bmatrix} -\frac{1}{2}(x_1^{(1)}(1) + x_1^{(1)}(2)) & x_2^{(1)}(2) & \dots & x_N^{(1)}(2) \\ -\frac{1}{2}(x_1^{(1)}(2) + x_1^{(1)}(3)) & x_2^{(1)}(3) & \dots & x_N^{(1)}(3) \\ \vdots & \vdots & \ddots & \vdots \\ -\frac{1}{2}(x_1^{(1)}(n-1) + x_1^{(1)}(n)) & x_2^{(1)}(n) & \dots & x_N^{(1)}(n) \end{bmatrix} \quad (5-16)$$

$$Y = [x_1^{(0)}(2) \ x_1^{(0)}(3) \dots x_1^{(0)}(n)]^T \quad (5-17)$$

当 $x_i^{(1)}(i = 2, \dots, N, i \neq 1)$ 呈现出缓变，可以用区间不变值进行计算时，则 x_i 的时间(离散)近似关系式为：

$$\begin{aligned} \hat{x}_1^{(1)}(k+1) &= \left(x_1^{(0)}(1) - \frac{1}{a} \sum_{i=2}^N b_{i-1} x_i^{(1)}(k+1) \right) e^{-ak} + \frac{1}{a} \sum_{i=2}^N b_{i-1} x_i^{(1)}(k) \\ &+ 1) x_i^{(0)}(0) = x_i^{(0)}(1) \quad i = 1, 2, \dots, N \end{aligned} \quad (5-18)$$

若上述关系不成立，则只有通过 $x_1, x_2, \dots, x_n$ 的 N 个一阶微分方程联立求解。

(4) BP 神经网络

BP 神经网络是一种典型的多层前馈型(通常为三层)的人工神经网络^[7]。从结构上讲，它由输入层、隐含层和输出层组成，其中隐含层包含一层或多层，且各层内均有节点，相邻层节点之间由权值连接，但各层内的节点之间相互独立。BP 神经网络的结构简图如图 9 所示。

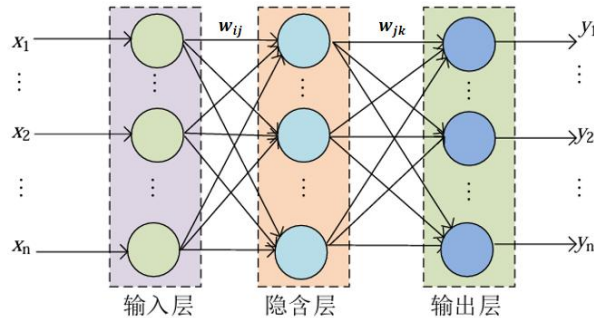


图 9 BP 神经网络结构图

图中, x_n 为输入的变量, i 、 j 、 k 分别为输入层、隐含层和输出层的节点数量, w_{ij} 与 w_{jk} 分别为层间的连接权值。

BP 神经网络的步骤具体可分为以下几个部分:

- (1) 网络拓扑结构的确定 (输入-隐含-输出) 及相关参数的初始化;
- (2) BP 神经网络的学习, 信息由输入层经隐含层传递到输出层, 经输出层计算后输出该过程网络的实际输出;
- (3) 计算网络数据实际输出与样本数据期望输出之间的误差, 并依据得到的误差信息反向传播到输入层, 同时调整各层之间的权值、阈值;
- (4) 循环迭代 (2)、(3) 两个过程, 逐步降低计算误差, 直到误差达到设定的目标误差或循环迭代次数达到设定的最大次数;
- (5) 获取到最优的权值、阈值;
- (6) 提取检测样本的输入信息, 借助第 (5) 步获取到的最优的权值、阈值, 便可计算出测试样本的预测输出。

其具体步骤的流程如图 10 所示:

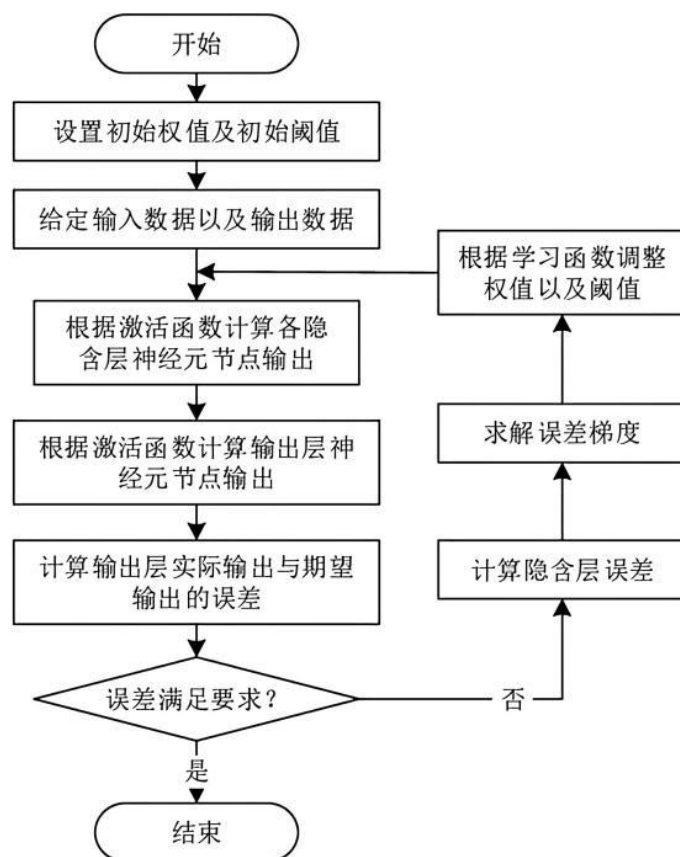


图 10 BP 神经网络流程图

(5) LSTM 神经网络

长短记忆神经网络(Long and Short Term Memory,LSTM)在 RNN 神经网络的基础上引进遗忘门, 不但可以很好的处理序列数据问题, 而且有效的避免了 RNN 神经网络因为保留过多的输入信息和训练信息引起的梯度爆炸或梯度消失的问题^[8]。LSTM 神经网络的基本单元包括遗忘门 F_t 、输入门 I_t 及输出门 O_t , 如图 11 所示。

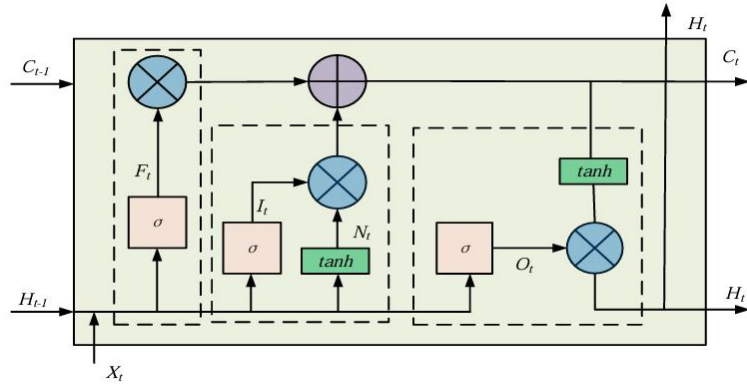


图 11 LSTM 神经网络结构图

F_t 通过忘记的方式丢弃不重要的信息以提高神经网络的计算能力。

$$F_t = \sigma(W_F[H_{t-1}, X_t] + b_F) \quad (5-19)$$

其中, σ 为Sigmoid函数, W 为权重, b 为偏置项, H_{t-1} 为上一时刻单元输出, X_t 为当前输入。输入门对上一时刻隐含层信息和当前时刻的输入信息进行计算, 得到更新信息 N_t 。

$$I_t = \sigma(W_I[H_{t-1}, X_t] + b_I) \quad (5-20)$$

$$N_t = \text{Tanh}(W_C[H_{t-1}, X_t] + b_C) \quad (5-21)$$

其中, Tanh 为状态激活函数。然后, 将遗忘门与输入门结合, 计算得到更新的单元状态 C_t 。

$$C_t = F_t C_{t-1} + I_t N_t \quad (5-22)$$

最后, 输出门将单元内重要信息进行输出并与经过状态激活的更新单元状态相乘得到最终输出结果 H_t 。

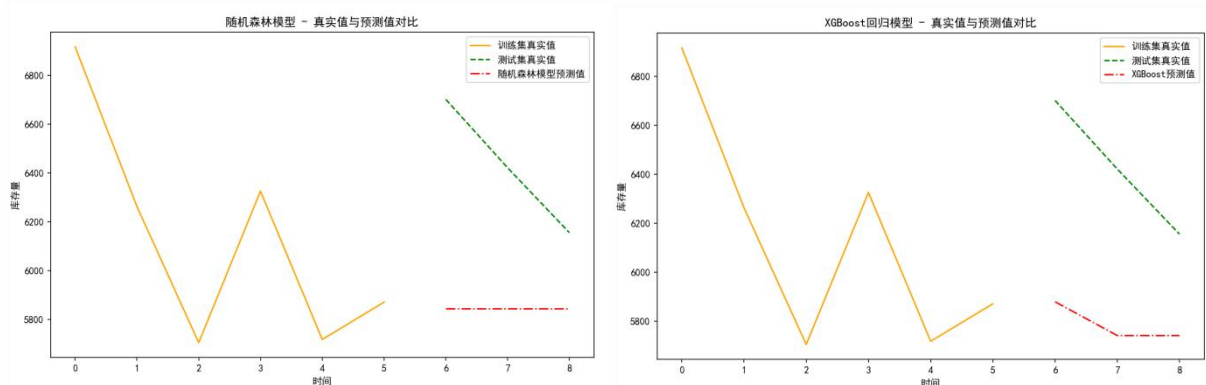
$$O_t = \sigma(W_O[H_{t-1}, X_t] + b_O) \quad (5-23)$$

$$H_t = O_t \text{Tanh}(C_t) \quad (5-24)$$

因此, LSTM 神经网络使用遗忘门对以往的神经单元状态进行选择记忆, 遗忘门将不重要的信息进行丢弃减少了神经网络整体的计算量, 同时有效避免了梯度爆炸或梯度消失。

5.2.2 子问题 a) 模型的求解

基于附件 1 的相关数据, 以 80%的数据为训练集, 以 20%的数据为测试集依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络^[2], 得到五个模型的训练集真实值、测试集真实值以及各模型预测值, 如图 12 所示:



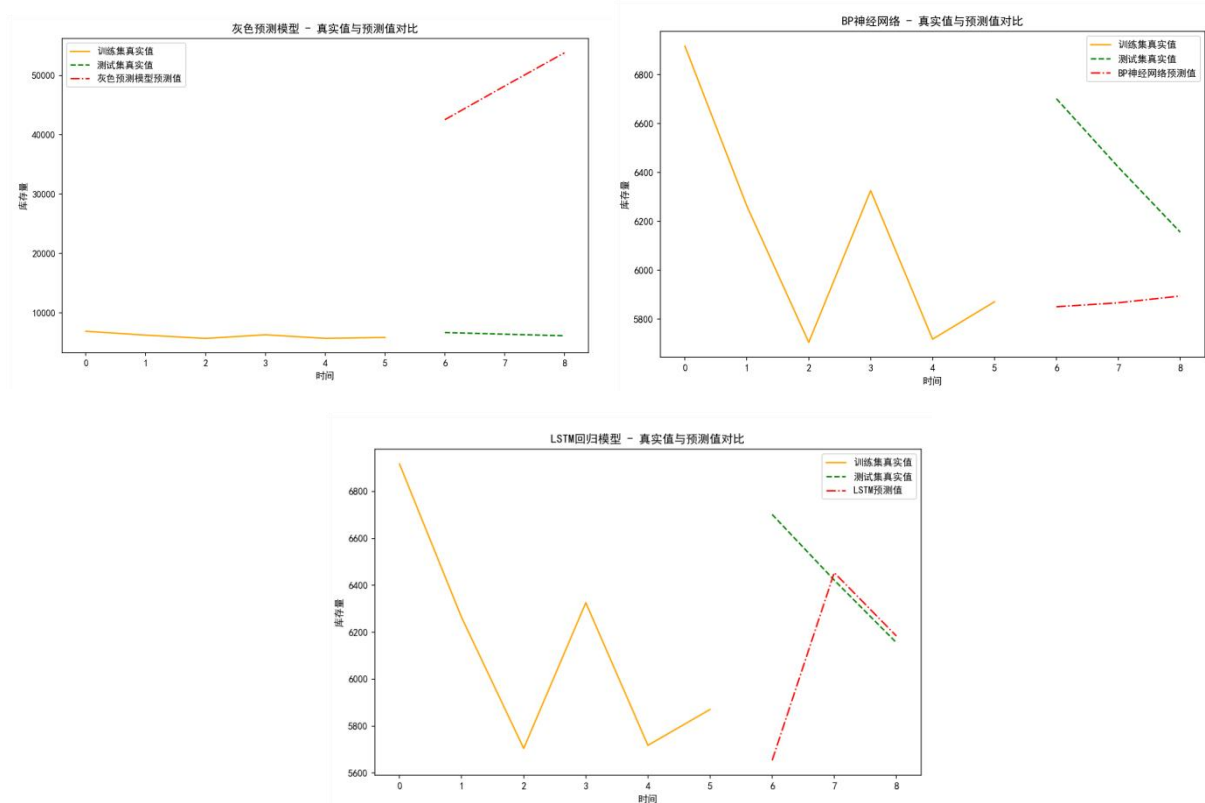


图 12 五种模型的训练集、测试集真实值以及各模型库存量预测值

由图 12 可知，对于随机森林模型，预测值同样与训练集真实值存在一定偏差。在某些时间点上，预测值与真实值非常接近，而在其他时间点上则存在较大差异。这表明随机森林模型在某些情况下能够较好地捕捉到数据的趋势，但在其他情况下可能会产生较大的误差。

对于 XGBoost 模型，预测值与训练集真实值之间存在一定的偏差。在时间序列的早期阶段，预测值与真实值较为接近，但随着时间的推移，预测值开始偏离真实值，同样显示出一定的误差。

对于灰色预测模型，预测值与训练集真实值之间存在一定的偏差。在时间序列的早期阶段，预测值与真实值较为接近，但随着时间的推移，预测值开始偏离真实值，显示出一定的误差。

对于 BP 神经网络，预测值与训练集真实值之间的偏差相对较小。在整个时间序列中，预测值与真实值都保持了较好的一致性。这表明 BP 神经网络在预测库存量方面具有较高的准确性。

对于 LSTM 神经网络，预测值与训练集真实值之间的偏差也相对较小。与 BP 神经网络类似，LSTM 神经网络在整个时间序列中都保持了较好的一致性，显示出较高的预测准确性。

为了解各模型预测的相对准确性，进一步衡量各模型预测值与实际值之间的百分比误差，平均绝对百分比误差（Mean Absolute Percentage Error, MAPE），它是一种常用的预测模型评估指标，主要用于衡量预测值与实际值之间的相对误差大小。MAPE 值的计算公式为：

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| * 100\% \quad (5-25)$$

MAPE 值的取值范围是 $[0, +\infty)$ ，其值越接近于 0，表示预测值与实际值越接近，预测模型的准确度越高。在实际应用中，MAPE 值小于 10%通常被认为是比较好的预测结果，MAPE 值在 10% – 20%之间时，预测的精度还可以接受。但如果 MAPE 值大于 20%，则预测的效果可能不太理想，需要进一步改进预测模型。各模型 MAPE 值对比如图 13 所示。

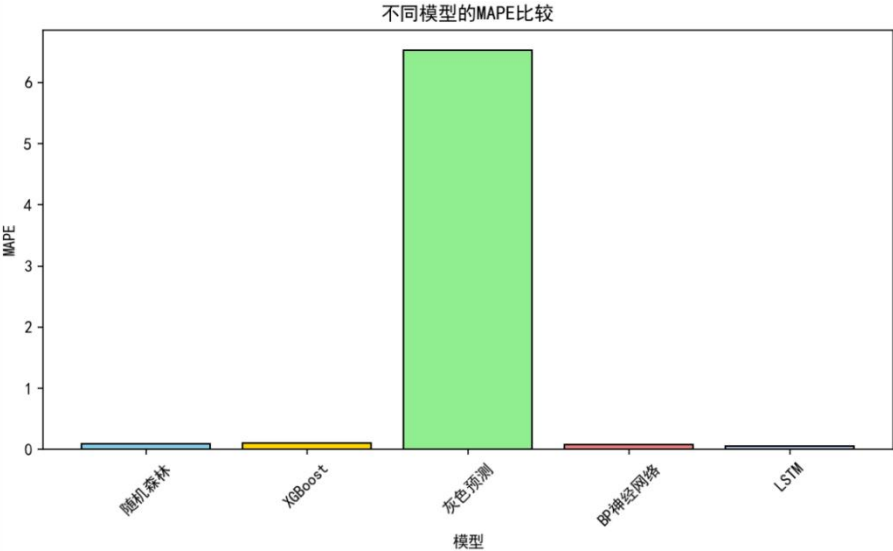


图 13 附件 1 五种模型的 MAPE 比较

从图 13 中可以看出，LSTM 神经网络的 MAPE 值最低，说明其预测性能最好；其次是 BP 模型、随机森林模型、XGBoost 模型。而灰色预测模型的 MAPE 值最高，表明其预测性能最差。

再基于附件 1 的相关数据，以 80%的数据为训练集，以 20%的数据为测试机依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络，得到五个模型的评估参数对比，如表 4 所示：

表 4 附件 1 五种模型评估参数对比

模型	数据集	准确率	召回率	精确率	F1-score
随机森林模型	训练集	0.688	0.688	0.582	0.581
	测试集	0.712	0.712	0.421	0.521
XGBoost 模型	训练集	0.725	0.725	0.649	0.623
	测试集	0.693	0.693	0.581	0.495
灰色预测模型	训练集	0.622	0.622	0.643	0.527
	测试集	0.521	0.521	0.552	0.516
BP 神经网络	训练集	0.676	0.676	0.525	0.518
	测试集	0.713	0.713	0.478	0.435
LSTM 神经网络	训练集	1	1	1	1
	测试集	0.8	0.8	0.836	0.784

由表 4 来看，综合准确率、召回率、精确率、F1 四项指标来看，LSTM 神经网络表现最优，准确率、召回率和精确率均在 0.8 以上，且准确率达 0.836。

通过图 12~图 13 及表 4 分析表明 LSTM 神经网络模型与其它模型相比不仅预测性能

最好，还能显示出较高的预测准确性，因此本文选择利用 LSTM 神经网络模型对附件 1 的库存量进行预测。

首先对附件 1 的数据进行归一化处理，其主要目的是将数据缩放到一个合适的范围内，便于神经网络的训练和优化。对数据进行归一化可以改善梯度下降、加速收敛、提高模型的泛化能力。

本文选择 Z-score 标准化方法对数据进行处理，Z-score 标准化（也称为标准差标准化）是一种常见的数据归一化方法，其基本思想是将原始数据转换为标准正态分布，即均值为 0，标准差为 1，计算公式为

$$Z = \frac{X - \bar{X}}{s} \quad (5-27)$$

是一个实测值与平均数的差再除以标准差的过程。

通过对附件 1 数据的分析，发现库存量千差万别，这样就会导致样本数据中标签值 min 和 max 差距非常大，如图 14 所示。

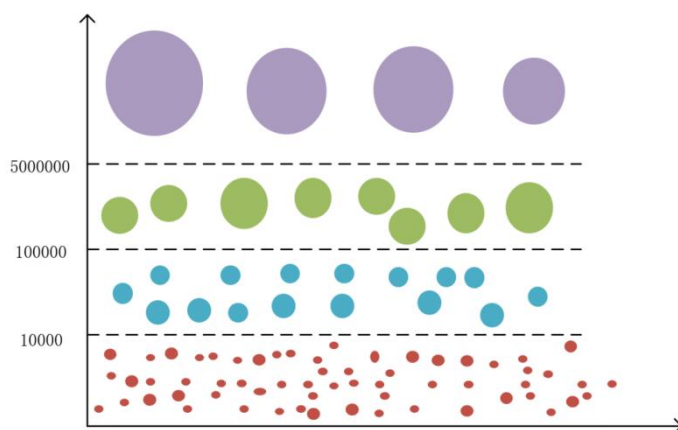


图 14 附件 1 数据量维度

y轴表示商品维度标签值的库存数据量级，库存数据量较小的组合维度占有很大一部分，而库存数据量较大的组合维度相对较少，仅凭数据归一化手段，数据压缩的效果非常不明显，如果强行一起训练会互相影响，导致模型难以拟合，训练中的损失函数如图 15 所示，训练时模型无法正常拟合，损失值不能够正常下降，训练出的模型效果可想而知，无法支持正常的预估功能。

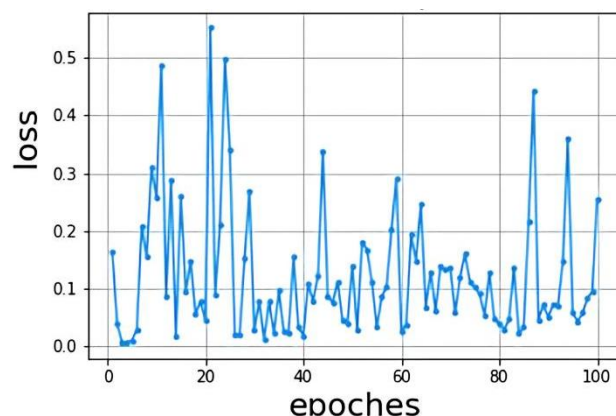


图 15 附件 1 损失函数结果

所以，以库存样本的标签值为依据，使用 K-means 算法对不同维度的库存进行数据聚类，将近似类别的维度放在一起进行训练，提高模型的拟合速度和泛化能力。选择“肘部法则”来确认本数据集的最佳分类数，也就是 K-means 的簇数 K 的具体值。随着 K 的增加，聚类效果不断提高，但是当 K 到达某个值的时候，聚类效果的提高变得越来越慢，此时再增加 K 已经不能明显提高聚类效果，因此这个点就是最佳的 K 值。具体过程如下：

①对数据集进行 K-Means 聚类，分别尝试不同的簇数 K 。

②对于每个簇数 K ，计算该簇数下的 SSE（Sum of Squared Errors），即每个数据点到其所属簇中心的距离的平方和，保存 SSE 到数组。

③使用 numpy 库中的 diff 函数计算相邻两点的差异并除以最大值，得到变化率。然后，我们使用 numpy 库中的 argmax 函数找到最大斜率的位置，加上 n 即为肘部点的位置。

④ n 的取值要考虑自身数据集，一般来说， K 的值至少要为 2，不然没有分类意义，而且 argmax 计算出来的是数组的索引位置，小于真实位置 1 个点位，综合考虑，我们将 n 设置为 2，SSE 随簇数变化的曲线如图 16 所示。

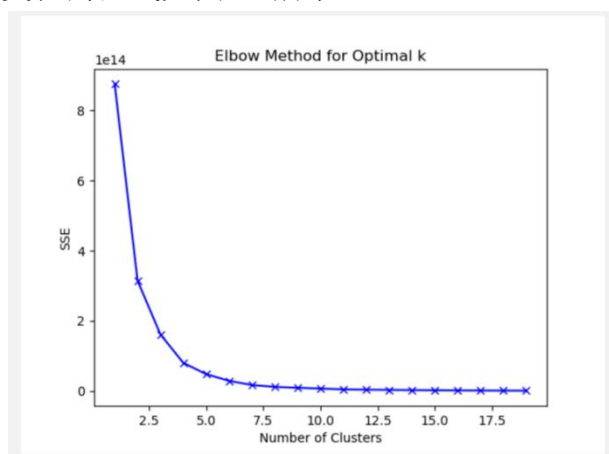


图 16 SSE 随簇数变化曲线

经过 K 值选择后，我们将样本数据集进行聚类得到如图 17 所示结果， y 轴为商品维度的标签值， x 轴为是商品维度出现在样本数据集数组中的索引位置。其中最右侧最高的点，明显是没有任何维度交叉的全部商品库存，其他位置都是经过维度交叉后的库存，最终我们将其分成了 3 份进行训练。

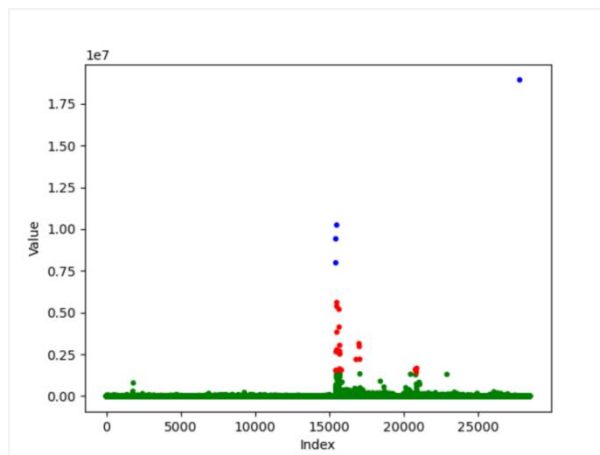


图 17 数据集聚类结果

经过聚类处理后，模型训练时损失函数输出的损失值如图 18 所示，出现了正常的损失值下降的过程，并逐渐趋于稳定。

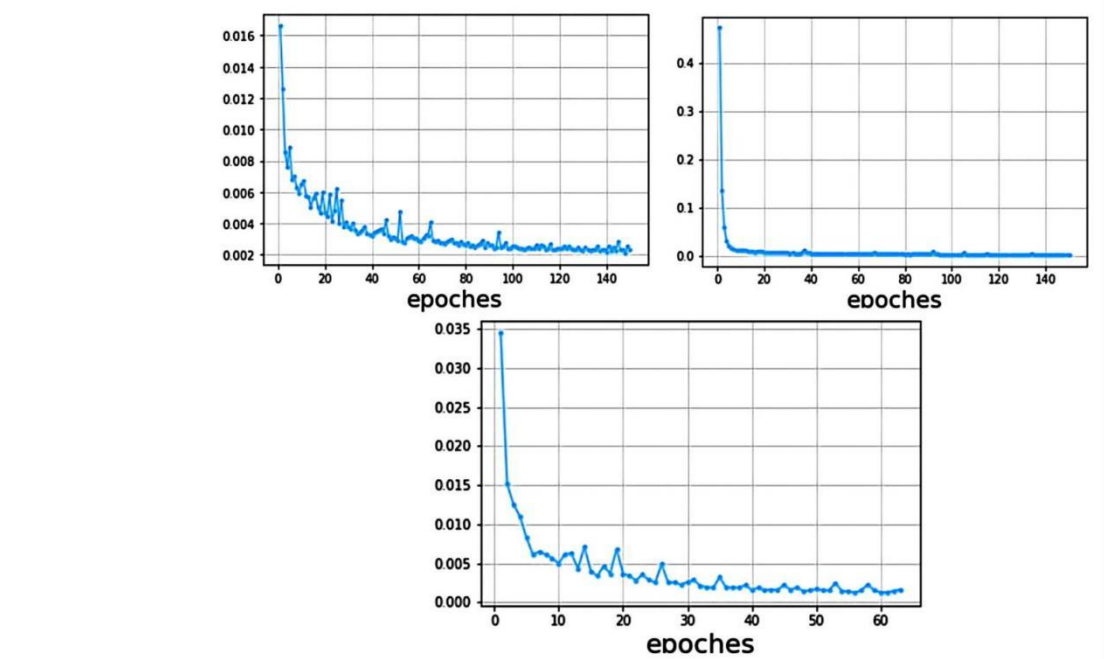


图 18 附件 1 聚类处理后损失函数结果

利用构建的 LSTM 神经网络模型对附件 1 的库存量进行未来 3 个月（7-9 月）的预测，350 个品类的未来 3 个月预测结果见附件 1 中的 result 表格，其中问题 1 所需品类月库存量预测结果如表 5 所示。

表 5 月库存量预测结果

	7 月库存量	8 月库存量	9 月库存量
category1	6205	6245	6227
category31	187	158	136
category61	236244	236244	236244
category91	6294	6006	5745
category121	74989	74989	74989
category151	115715	117206	117206
category181	503	503	503
category211	7435	7435	7435
category241	53196	53196	53196
category271	14758	14758	14758
category301	2229	2822	2397
category331	66888	83975	105368

5.3 子问题 b) 的建模与求解

5.3.1 子问题 b) 模型的建立

本文参考问题 1 的子问题 a)，以上述数据处理过的附件 2 中的日期为自变量，以销售量为因变量，依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和

LSTM 神经网络。

5.3.2 子问题 b) 模型的求解

基于附件 2 的相关数据，以 80% 的数据为训练集，以 20% 的数据为测试集依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络，得到五个模型的训练集真实值、测试集真实值以及各模型预测值，如图 19 所示：

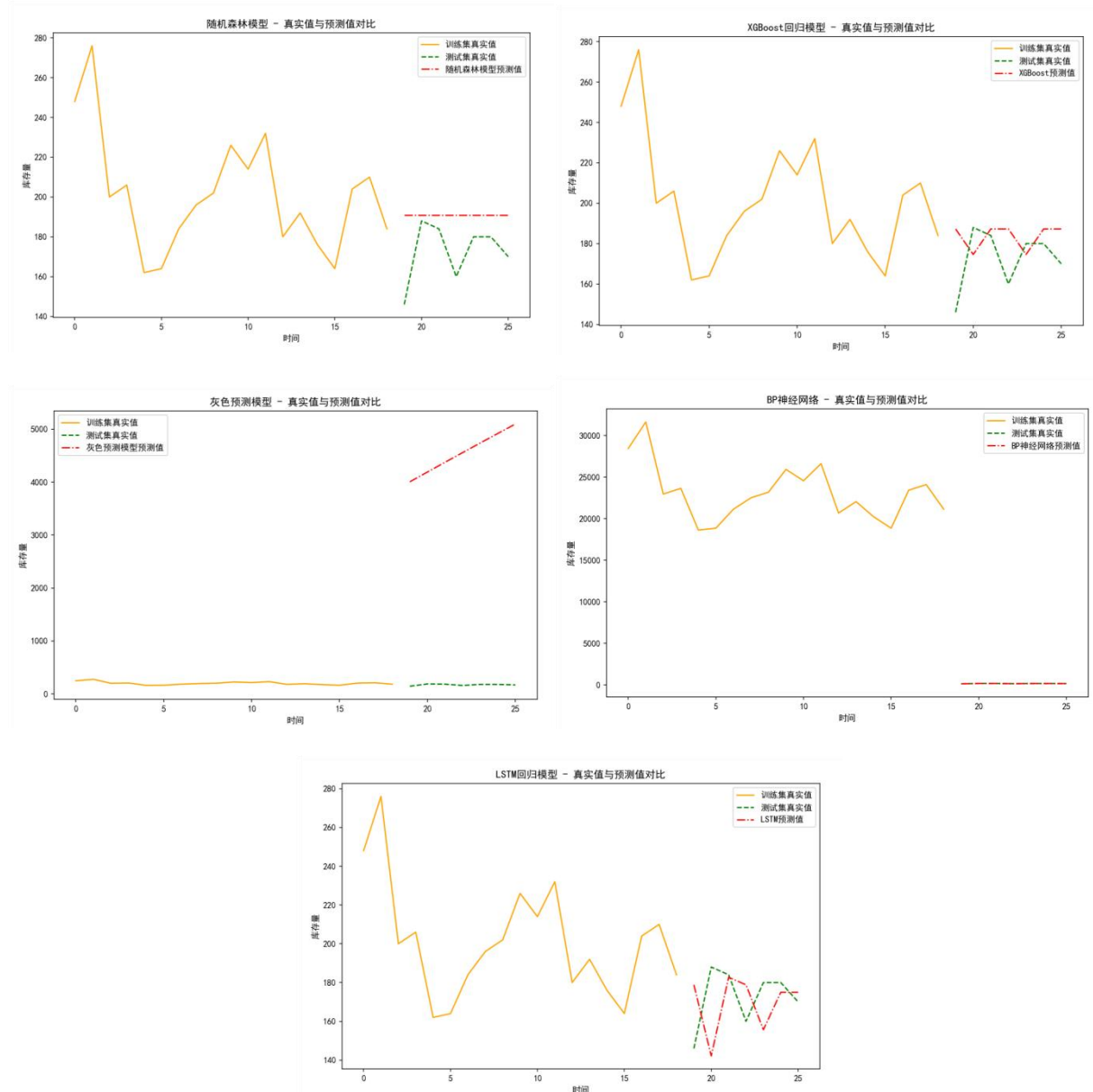


图 19 五种模型的训练集、测试集真实值以及各模型销售量预测值

由图 19 可知，对于随机森林模型，预测值与训练集真实值之间存在一定的偏差，特别是在时间序列的早期阶段。随着时间的推移，预测值开始偏离真实值，显示出一定的误差。

对于 XGBoost 模型，与随机森林模型相比，XGBoost 回归模型的预测值更接近于训练集真实值，尤其是在时间序列的后期阶段。这表明 XGBoost 回归模型可能具有更好的泛化能力和稳定性。

对于灰色预测模型，预测值与训练集真实值之间的偏差较大。这可能是由于灰色预测模型对数据的变化趋势捕捉不够准确所导致的。

对于 BP 神经网络，与前三中模型相比，BP 神经网络的预测值更接近于训练集真实值，尤其是在时间序列的后期阶段。这表明 BP 神经网络可能具有更强的学习能力和适应性，能够更好地捕捉到数据的变化趋势。

对于 LSTM 神经网络，LSTM 模型的预测值与训练集的真实值存在一定的偏差。在某些时间点上，预测值与真实值非常接近，而在其他时间点上则存在较大的差异。

这五种模型在销售量预测方面都表现出了一定的能力，但也存在不同程度的误差。其中，BP 神经网络的预测性能相对较好，而灰色预测模型的预测性能相对较差。

为了解各模型预测的相对准确性，进一步衡量各模型预测值与实际值之间的百分比误差，平均绝对百分比误差（Mean Absolute Percentage Error, MAPE），它是一种常用的预测模型评估指标，主要用于衡量预测值与实际值之间的相对误差大小。MAPE 值的计算公式为：

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| * 100\% \quad (5-28)$$

MAPE 值的取值范围是 $[0, +\infty)$ ，其值越接近于 0，表示预测值与实际值越接近，预测模型的准确度越高。在实际应用中，MAPE 值小于 10%通常被认为是比较好的预测结果，MAPE 值在 10% – 20%之间时，预测的精度还可以接受。但如果 MAPE 值大于 20%，则预测的效果可能不太理想，需要进一步改进预测模型。各模型 MAPE 值对比如图 20 所示。

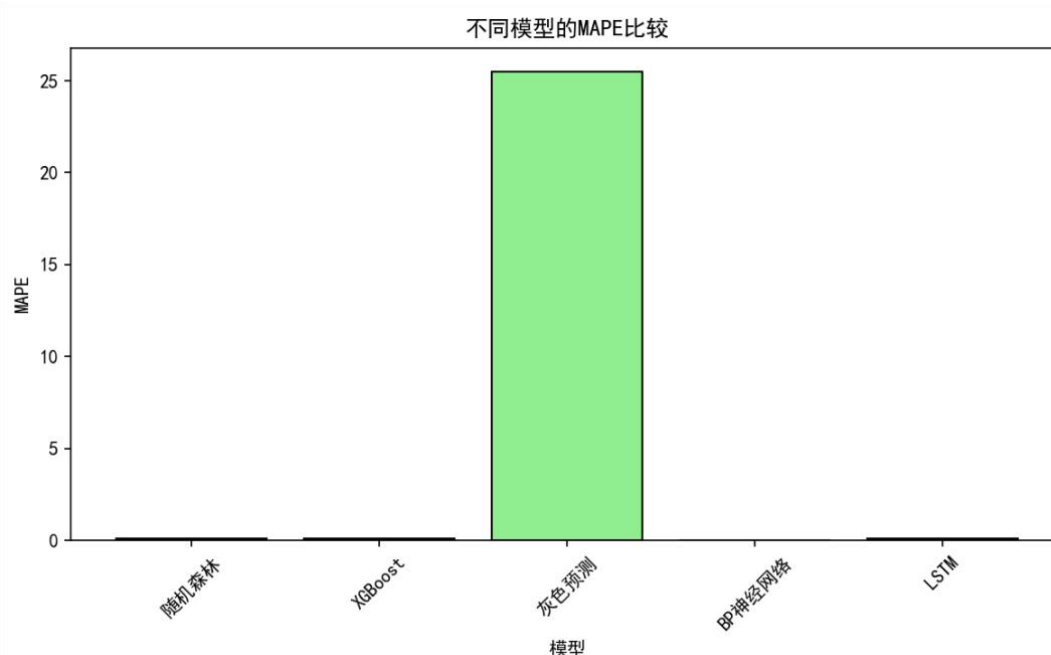


图 20 附件 2 五种模型的 MAPE 比较

从图 20 中可以看出，BP 神经网络的 MAPE 值最低，说明其预测性能最好；其次是 LSTM 模型、XGBoost 模型、随机森林模型。而灰色预测模型的 MAPE 值最高，表明其预测性能最差。

再基于附件 2 的相关数据，同样以 80%的数据为训练集，以 20%的数据为测试机依次构建随机森林模型、XGBoost 模型、灰色预测模型、BP 神经网络和 LSTM 神经网络，得到五个模型的评估参数对比，如表 6 所示：

表 6 附件 2 五种模型评估参数对比

模型	数据集	准确率	召回率	精确率	F1-score
随机森林模型	训练集	0.692	0.692	0.537	0.527
	测试集	0.569	0.569	0.518	0.508
XGBoost 模型	训练集	0.727	0.727	0.681	0.611
	测试集	0.741	0.741	0.674	0.603
灰色预测模型	训练集	0.575	0.575	0.419	0.402
	测试集	0.553	0.553	0.406	0.396
BP 神经网络	训练集	1	1	1	1
	测试集	0.814	0.814	0.842	0.786
LSTM 神经网络	训练集	0.738	0.738	0.695	0.572
	测试集	0.714	0.714	0.644	0.514

由表 6 来看，综合准确率、召回率、精确率、F1 四项指标来看，BP 神经网络表现最优，准确率、召回率和精确率均在 0.8 以上，且准确率达 0.814。

通过图 19~图 20 及表 6 分析表明 BP 神经网络模型与其它模型相比不仅预测性能最好，还能显示出较高的预测准确性，因此本文选择利用 BP 神经网络模型对附件 2 的销售量进行预测。

上述已经对附件 2 进行数据清洗，因此现需对原始进行差分以消除趋势性和季节性影响，使数据符合 BP 神经网络训练要求，差分结果如图 21 所示。

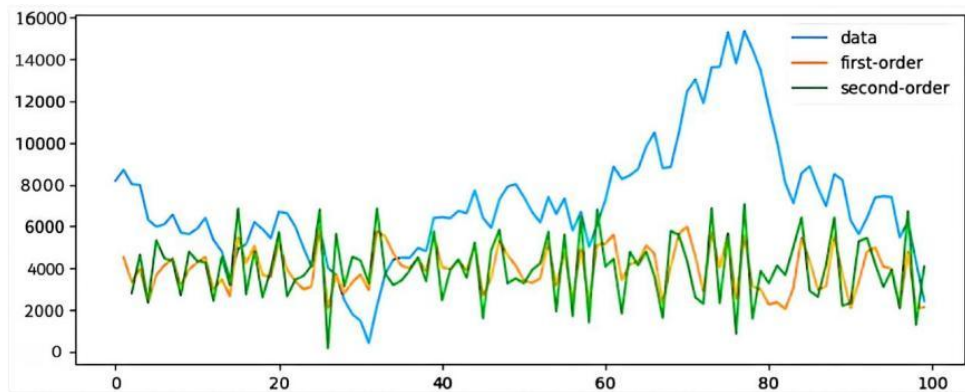


图 21 附件 2 差分结果

为方便网络训练和优化，将附件 2 数据范围归一化到 0~1 之间，采用 Z-score 标准化，它是通过减去均值并处以标准差来调整公式，核心公式为

$$x' = \frac{x - \mu}{\delta} \quad (5-29)$$

其中， μ 为均值， δ 为标准差。将原始数据分布图与标准化后的数据分布进行比较，如图 22 所示。

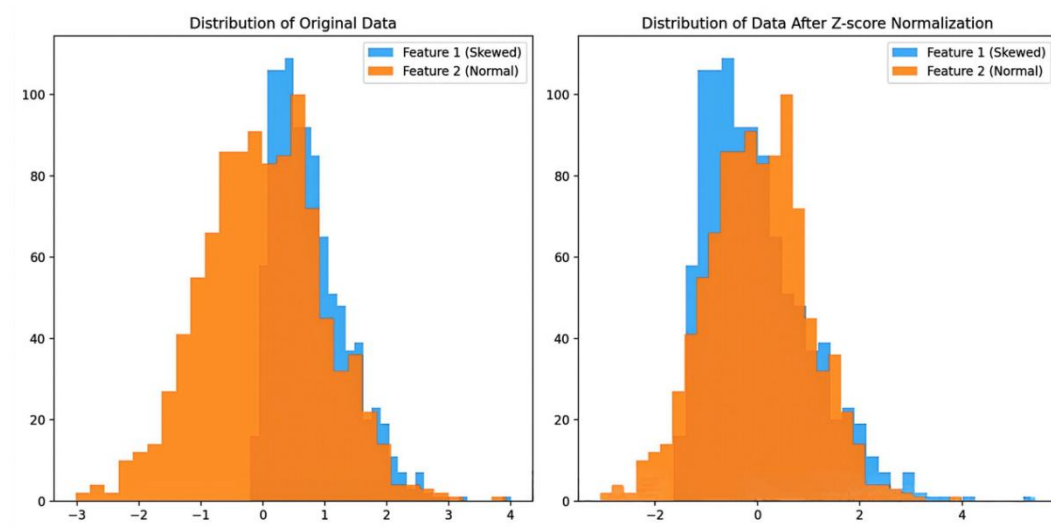


图 22 Z-score 标准化结果

从图 22 中发现，原始数据表现出明显的偏态分布，而标准化后的数据为正态分布。并且，在 Z-score 标准化后，分布变得均值为 0，标准差为 1，原始分布形状仍然保留，但已经在相同的尺寸上。

选择合适的网络结构，包括输入层、隐藏层和输出层的神经元数量。输入层节点数应与特征数量相匹配，输出层节点数为 1，因为我们预测的是销量这一个连续值。以 80% 的数据为训练集，以 20% 的数据为测试集。使用训练集数据进行前向传播，计算输出层的预测结果。再计算预测结果和实际标签之间的误差。通过反向传播算法调整网络权重和偏置，以减少误差。重复上述过程多次（迭代），直到网络性能不再显著提升或达到预设的迭代次数（1000 次），如图 23 所示。

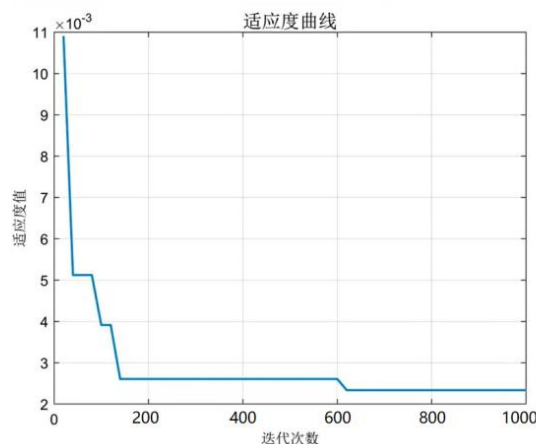


图 23 模型迭代次数

假设训练集有 n 期数据，测试集有 v 期数据， y_t 表示实际数据， e_t 表示预测误差， $\hat{y}_t$ 表示预测值， $\bar{y}_t$ 表示平均值，有以下几种衡量预测精度的指标，最后使用测试集数据评估模型的预测性能，模型评估结果如表 7 所示。

$$\text{均方误差 (MSE)} = \frac{1}{v} \sum_{t=1}^v e_t^2 \quad (5-30)$$

$$\text{均方根误差 (RMSE)} = \sqrt{\frac{1}{v} \sum_{t=1}^v e_t^2} \quad (5-31)$$

$$\text{平均绝对误差 (MAE)} = \frac{1}{v} \sum_{t=1}^v |e_t| \quad (5-32)$$

$$\text{平均绝对百分比误差 (MAPE)} = \frac{1}{v} \sum_{t=1}^v \left| \frac{e_t}{y_t} \right| * 100\% \quad (5-33)$$

$$\text{决定系数 (R}^2\text{)} = 1 - \frac{\sum_{t=1}^v (y_t - \hat{y}_t)^2}{\sum_{t=1}^v (\hat{y}_t - \bar{y}_t)^2} \quad (5-34)$$

表 7 模型评估结果

	MSE	RMSE	MAE	MAPE	R ²
训练集	0.007	0.067	0.025	0.012	0.868
交叉验证集	0.012	0.084	0.031	0.011	0.861
测试集	0.009	0.192	0.085	0.014	0.894

根据表 7 可知，各误差均在合适范围内，表明该模型准确度较高，因此该模型预测效果较好，可使用训练好的模型对未来的销量进行预测，350 个品类的未来每天的预测结果见附件 1 中的 result 表格，其中问题 1 所需品类日销量预测结果如表 8 所示。

表 8 日销量预测结果

	7.1	7.11	7.21	7.31	8.11	8.21	8.31	9.11	9.21
category1	29	27	25	23	22	26	22	30	28
category31	0	0	0	0	0	0	0	0	0
category61	2301	2204	2154	2235	2057	2015	1950	2235	2076
category91	4	4	2	4	3	2	3	3	3
category121	1173	1124	992	945	870	815	790	933	867
category151	1751	1677	1639	1701	1565	1533	1484	1701	1580
category181	4	4	6	6	5	3	3	5	5
category211	29	28	28	29	26	26	25	29	27
category241	681	652	638	662	609	596	577	662	615
category271	189	181	181	180	165	171	165	197	183
category301	37	35	34	36	33	32	31	36	33
category331	5802	5556	6721	8597	7911	9528	11312	15876	14747

6 问题 2 的模型建立与求解

6.1 数据预处理

由于问题 2 需要分析各品类之间基于问题 1 的预测结果建立规划模型。问题 1 所得为月库存量的数据集，为方便计算，因此首先需要将问题 1 所得每天销售量的数据集按每月进行归类，归类结果见附件 3。

6.2 问题 2 的建模与求解

6.2.1 问题 2 模型的建立

根据电商企业分仓的实际运作模式，电商企业需要利用历史历史库存量及销量来获得最优的商品分仓决策，进而为后续的管理效率提供支持^[9]。基于此，如图 24 所示（图中问号代表商品分仓问题，表示如何将商品划分到仓库），本题研究的商品分仓优化问题可被表述为：给定由 n 个不同的商品构成的商品集合 $V = \{v_1, v_2, \dots, v_n\}$ 和 m 个不同的仓库构成的仓库集合 $W = \{w_1, w_2, \dots, w_m\}$ ，如何合理的将 V 划分到 W （其中每个子集表示一个仓库中存储的商品），使得每个品类都能放分别在各个仓库中。

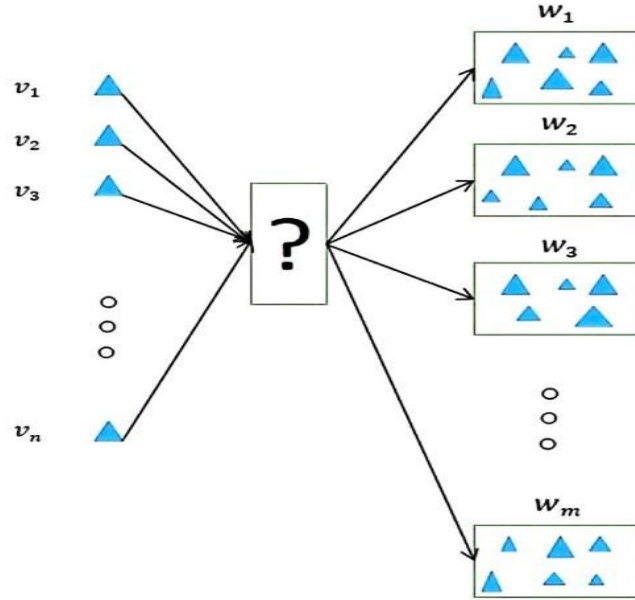


图 24 商品分仓问题示例

（1）目标函数的建立

确定模型变量，令第 i 个产品对第 j 个仓库的状态为 x_{ij} ，

$$x_{ij} = \begin{cases} 0, & \text{第 } i \text{ 个产品不在第 } j \text{ 个仓库储存} \\ 1, & \text{第 } i \text{ 个产品在第 } j \text{ 个仓库储存} \end{cases} \quad (6-1)$$

其中， $i = 1, 2, 3, \dots, 350$ ； $j = 1, 2, \dots, 140$ 。

对于所有产品的总成本为 P ，各仓库使用单价为 p_j ，使用各仓库的时长为 t_j ， t_j 为商品放置时间的长短，由于每个月的天数不一样，因此， t_j 根据实际情况进行取值，所以目标函数可表示为：

$$\min P = \sum_{j=1}^{j=140} p_j t_j h_j \quad (6-2)$$

目标函数解释说明：目的是让每个商品都有位置存放，并且仓库数量及时长最少的情况下，总成本最少。

（2）约束条件的确定

根据问题 2 的要求，在寻找最优“一品一仓”方案的情况下，拥有下述约束条件：

①同一产品储存到同一仓库：

$$\sum_{j=1}^{140} x_{ij} = 1, (i = 1, 2, 3, \dots, 350 \ j = 1, 2, \dots, 140) \quad (6-3)$$

②令第*i*个产品每个月的销量为*B_i*，对于每个产品的仓库储存的状态为*h_j*：

$$h_j = \begin{cases} 0, & \sum_{i=1}^{350} x_{ij} B_i = 0 \\ 1, & \sum_{i=1}^{350} x_{ij} B_i = 1 \end{cases} \quad (i = 1, 2, 3, \dots, 350 \ j = 1, 2, \dots, 140) \quad (6-4)$$

③令第*j*个仓库的仓容为*D_j*，对于各单仓储存而言，不能超过各单仓仓容：

$$\sum_{i=1}^{350} x_{ij} B_i \leq D_j, (i = 1, 2, 3, \dots, 350 \ j = 1, 2, \dots, 140) \quad (6-5)$$

④第*j*个仓库的产能为*C_j*，对于各单仓产能而言，不能超过各单仓产能上限：

$$\sum_{i=1}^{350} x_{ij} B_i \leq C_j, (i = 1, 2, 3, \dots, 350 \ j = 1, 2, \dots, 140) \quad (6-6)$$

(3) 建立“一品一仓”方案模型

根据上述的目标函数与约束条件，建立对应模型：

$$\begin{aligned} \min P &= \sum_{j=1}^{140} p_j t_j h_j \\ \text{s. t} &\begin{cases} \sum_{j=1}^{140} x_{ij} = 1 \\ h_j = \begin{cases} 0, & \sum_{i=1}^{350} x_{ij} B_i = 0 \\ 1, & \sum_{i=1}^{350} x_{ij} B_i = 1 \end{cases} \quad \left(\begin{array}{l} i = 1, 2, 3, \dots, 350 \\ j = 1, 2, \dots, 140 \end{array} \right) \\ \sum_{i=1}^{350} x_{ij} B_i \leq D_j \\ \sum_{i=1}^{350} x_{ij} B_i \leq C_j \end{cases} \end{aligned} \quad (6-7)$$

6.2.2 问题 2 模型的求解

将“一品一仓”问题看作线性规划问题，因此可以采用分枝定界法进行求解，该方法不但能求解纯整数规划问题，还能处理混合整数规划问题。分枝定界法是由杰出学者理查德·卡普首创，他曾成功解决含有 65 个城市的旅行商问题，并创下了当时的记录。该方法的核心思想在于将问题的可行解按照树状分支展开，然后在各个分支中寻找最佳解。每次分支后，对于那些已知不会超越已知可行解值的子集，将不再继续展开分支，因此许多子集（即搜索树上的许多节点）可以被排除在外，从而有效地缩小了搜索范围。这个过程一直持续，直到找到一个可行解，其值不超过任何子集的上界。因此，这种算法通常能够找到问题的最优解，具体算法流程如图 25 所示。

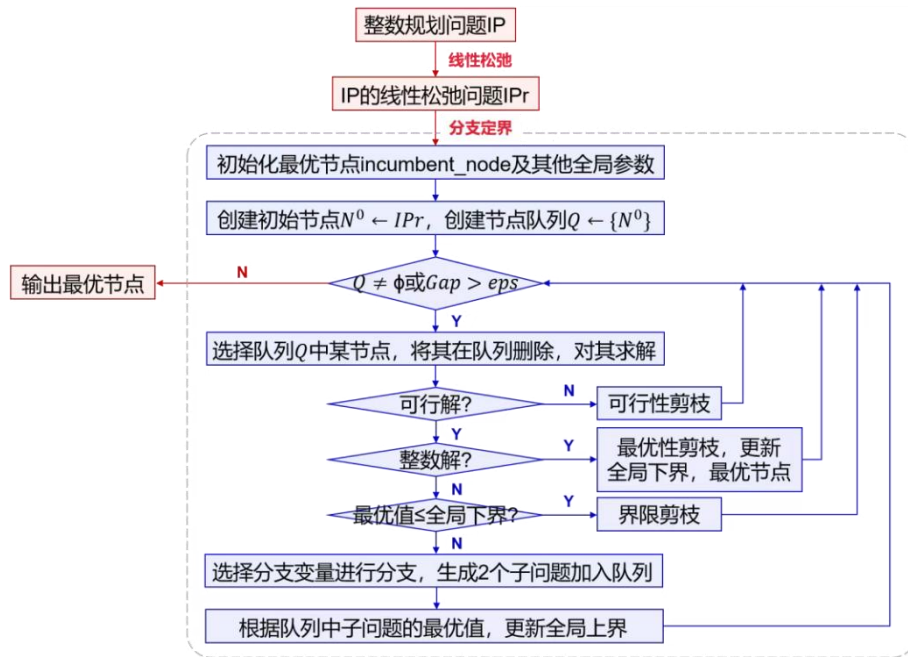


图 25 分支定界法算法流程图

分支定界法是解决运筹学中整数规划常用方法之一^[10]，但其理论求解过程多数要反复结合单纯形表才能得到，计算量很大。基于此，本文中的模型求解，我们利用 Lingo 软件来进行求解整数。

分支定界法是组合优化问题的有效求解方法，其步骤如表 9 所示：

表 9 分支定界法具体步骤

步骤	具体操作
1	如果问题的目标为最小化，则设定最优解的值 $z = \infty$ ；
2	根据分支法则，从尚未被洞悉节点（局部解）中选择一个节点，并在此节点的下一阶层中分为几个新的节点；
3	计算每一个新分支出来的节点的下限值；
4	对每一节点进行洞悉条件测试，若节点满足以下任意一个条件，则此节点可洞悉而不再被考虑，此节点的下限值大于等于 z 值。已找到在此节点中，具最小下限值的可行解；若此条件成立，则需比较此可行解与 z 值，若前者较小，则需更新 z 值，以此为可行解的值。此节点不可能包含可行解；
5	判断是否仍有尚未被洞悉的节点，如果有，则进行步骤二，如果已无尚未被洞悉的节点，则演算停止，并得到最优解；

导入上述处理的数据，利用 Lingo 以及 SCIP 求解器运行，同时，参照相关文献以及资料，设置下限和上限值分别为 1 和 3000，并在用分支定界法求解的过程中，恰当的从局部解中选择一个节点继续求最优解，并在此节点的下一阶层中分为几个新的节点，直至没有局部解进行下一个分支。节点队列采用先入后出的深优搜索，分支选择最不可行分支策略，剪枝使用查明的三种剪枝方法。基于数据集的前 7 个节点，其中第一个节点作为起点仓库，仓库容量根据附件 3 的数据集确定，使用分支定界算法求解至容差界限小于 0.001，用时 86 秒，迭代次数 3941 次，上下界收敛曲线如图 26 所示。

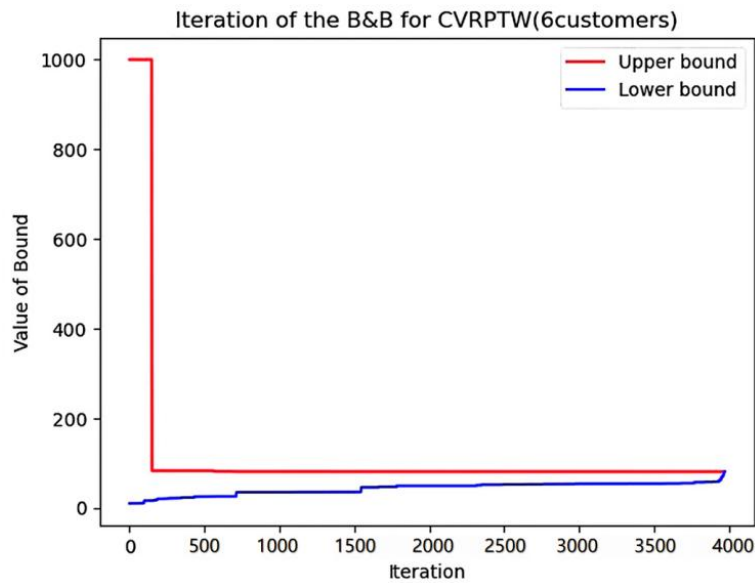


图 26 上下界收敛曲线

通过上述求解，得到各商品的仓库分配结果见附件 1 中的 result 表格，其中问题 2 所需商品分仓结果如表 10 所示。

表 10 “一品一仓”分仓方案

	warehouse
category1	warehouse41
category31	warehouse131
category61	warehouse60
category91	warehouse58
category121	warehouse20
category151	warehouse28
category181	warehouse21
category211	warehouse40
category241	warehouse133
category271	warehouse63
category301	warehouse133
category331	warehouse42

7 问题 3 的模型建立与求解

7.1 数据预处理

首先，我们分别读取问题一中所给“月库存预测结果”和“日销量预测结果”两个工作表的数据，读取题中所给的附件 3、附件 4、附件 5，分别读取了仓库数据、品类关联度数据和品类信息数据，并将它们分别存储在对应变量中。并对列名重命名与数据格式调整将数据转换为适当的格式，以便于我们后续的计算工作。接着，我们通过计算库存和销量

相关指标，通过获取 7、8、9 月预测值三列中的最大值计算了每个品类的最大库存量。由于品类关联度是作为最主要的优化目标之一，因此构建一个品类关联度矩阵，如图 27 所示，方便后续的目标函数构建，并对各指标设置权重系数。

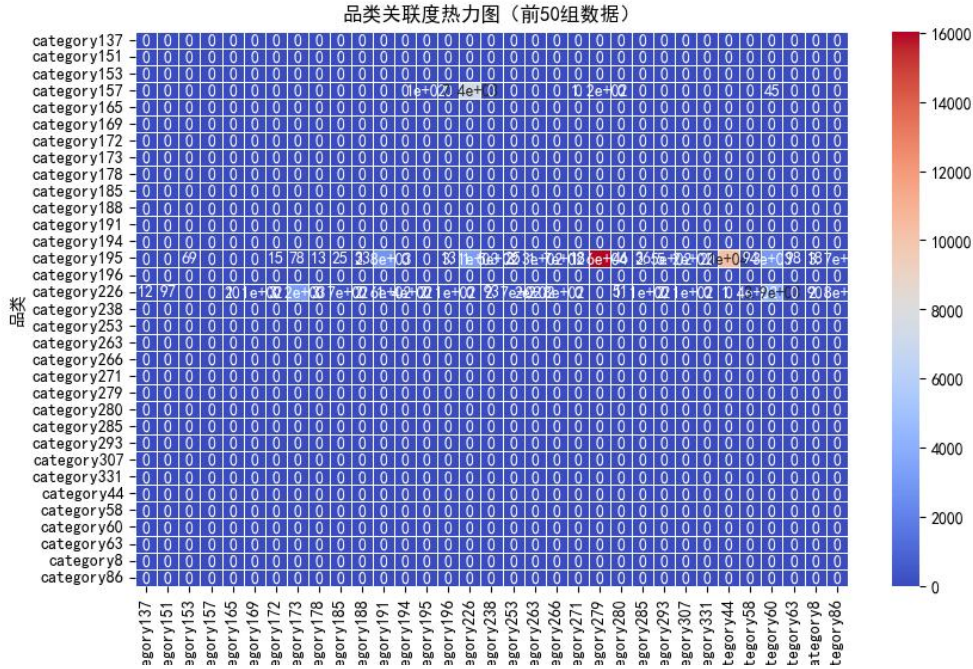


图 27 品类关联度矩阵

7.2 问题 3 的建模与求解

7.2.1 问题 3 模型的建立

(1) 目标函数的建立

该优化问题的目标是最大化仓库中品类的关联度总和，同时控制仓库容量和产能的松弛程度。通过添加松弛变量并在目标函数中给予惩罚权重，我们能够在满足约束条件的前提下提高品类关联度。向量目标函数表示为

$$F(x) = (f_1(x), f_2(x), f_3(x), f_4(x), f_5(x))^T \quad (7-1)$$

① 品类关联度 f_1 :

$$f_1 = \frac{1}{n} \sum_{k=1}^n \left(\frac{1}{|W_k|} \sum_{(i,j) \in W_k} S_{ij} \right) \quad (7-2)$$

其中 n 是仓库数量， W_k 是仓库 k 中分配的品类集合， $|W_k|$ 是仓库 k 中品类的数量。

② 产能利用率 f_2 :

$$f_2 = \frac{\sum_{k=1}^n \text{已使用容量}_k}{\sum_{k=1}^n \text{总容量}_k} \quad (7-3)$$

③ 仓能利用率 f_3 :

$$f_3 = \frac{\sum_{k=1}^n \left(\frac{\sum_{i \in W_k} \text{需求}_i}{C_k} \right)}{\sum_{k=1}^n 1} = \frac{1}{n} \sum_{k=1}^n \left(\frac{\sum_{i \in W_k} \text{需求}_i}{C_k} \right) \times 100\% \quad (7-4)$$

④总仓租成本 f_4 :

$$f_4 = - \left(\sum_{k=1}^n \text{租金}_k \right) \quad (7-5)$$

⑤品类分仓数 f_5 :

$$f_5 = - \left(\text{不同仓库中的品类数量} + \lambda \cdot \text{超过三个仓库的品类数} \right) \quad (7-6)$$

(2) 约束条件的确定

我们分别对仓容、产能、品类仓库分配进行限制。每个仓库的库存量不超过其仓容上限。为确保满足仓库容量限制，我们为每个仓库增加一个松弛变量，并在违反容量限制时对该松弛变量进行惩罚。每个仓库的出库量不超过其产能上限，同样需要松弛变量控制产能约束的违反程度，并且每个品类最多只能分配到 3 个仓库。

①每个仓库的出库量不超过其产能上限:

$$\sum_{i \in \text{品类}} x[i, j] \cdot \text{预测库存量}_i \leq \text{仓容上限}_j + \text{松弛变量}, \forall j \in \text{仓库} \quad (7-7)$$

②松弛变量控制产能约束的违反程度:

$$\sum_{i \in \text{品类}} x[i, j] \cdot \text{预测销量}_i \leq \text{每日最大出库量}_j + \text{松弛变量}, \forall j \in \text{仓库} \quad (7-8)$$

③每个品类最多只能分配到 3 个仓库:

$$\sum_{j \in \text{仓库}} x[i, j] \leq 3, \forall i \in \text{品类} \quad (7-9)$$

用于判断新生成的解是否被接受，最常用的接受准则是 Metropolis 准则，即当新解的目标函数值优于当前解时接受新解，否则以一定概率接受新解。具体公式如下:

$$C = c * \sum_{i \in N, j \in N} d_{ij} \leq \sum_{i \in N, j \in N} \quad (7-10)$$

$$d_{ij} \sum_{i=1}^n X_{ij} = Y_j, j = 1, 2, 3, \dots, n \quad (7-12)$$

$$\sum_{i=1}^n X_{ji} = Y_j, j = 1, 2, 3, \dots, n \quad (7-13)$$

(3) 建立“一品多仓”方案模型

根据上述的目标函数与约束条件，建立对应模型:

$$F(x) = (f_1(x), f_2(x), f_3(x), f_4(x), f_5(x))^T$$

$$\begin{aligned}
& \left. \begin{aligned}
& f_1 = \frac{1}{n} \sum_{k=1}^n \left(\frac{1}{|W_k|} \sum_{(i,j) \in W_k} S_{ij} \right) \\
& f_2 = \frac{\sum_{k=1}^n \text{已使用容量}_k}{\sum_{k=1}^n \text{总容量}_k} \\
& f_3 = \frac{\sum_{k=1}^n \left(\frac{\sum_{i \in W_k} \text{需求}_i}{C_k} \right)}{\sum_{k=1}^n 1} = \frac{1}{n} \sum_{k=1}^n \left(\frac{\sum_{i \in W_k} \text{需求}_i}{C_k} \right) \times 100\% \\
& f_4 = - \left(\sum_{k=1}^n \text{租金}_k \right) \\
& f_5 = - \left(\text{不同仓库中的品类数量} + \lambda \cdot \text{超过三个仓库的品类数} \right) \\
& \sum_{i \in \text{品类}} x[i, j] \cdot \text{预测库存量}_i \leq \text{仓库上限}_j + \text{松弛变量}, \forall j \in \text{仓库} \\
& \sum_{i \in \text{品类}} x[i, j] \cdot \text{预测销量}_i \leq \text{每日最大出库量}_j + \text{松弛变量}, \forall j \in \text{仓库} \\
& \sum_{j \in \text{仓库}} x[i, j] \leq 3, \forall i \in \text{品类} \\
& C = c * \sum_{i \in N, j \in N} d_{ij} \leq \sum_{i \in N, j \in N} \\
& d_{ij} \sum_{i=1}^n X_{ij} = Y_j, j = 1, 2, 3, \dots, n \\
& \sum_{i=1}^n X_{ji} = Y_j, j = 1, 2, 3, \dots, n
\end{aligned} \right\} \text{s. t.} \quad (7-14)
\end{aligned}$$

7.2.2 问题 3 模型的求解

模拟退火算法是一种基于概率的算法，模拟固体降温过程的最优化算法。其模拟的过程是首先将固体加温至某一温度，固体内部的粒子随温度上升慢慢变为无序的状态，内能增大，然后让其慢慢冷却，温度下降时，内部的粒子慢慢趋于有序，达到一种平衡态，最后达到常温时成为基态，此时内能减为最小，算法模拟这样一个过程期望能达到最优化的目的，如图 28 所示。多目标优化问题，是指在优化过程中存在多个目标函数需要同时最小化或最大化，这些目标函数之间往往存在冲突和矛盾，因此需要通过一定的方法来平衡这些目标之间的关系，从而得到一个满足所有目标的最优解集。

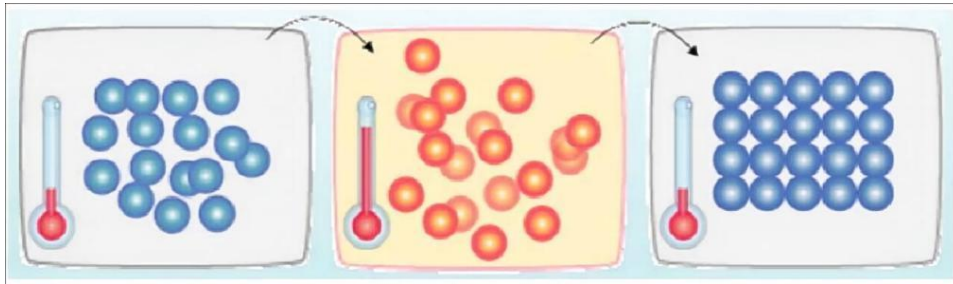


图 28 模拟退火算法示意图

多目标模拟退火算法在解决仓库存储问题时，特别是在考虑品类分配、件型、高级品类标签约束以及多个优化目标的情况下，展现出了其强大的全局搜索和优化能力，其具体

算法流程如图 29 所示。

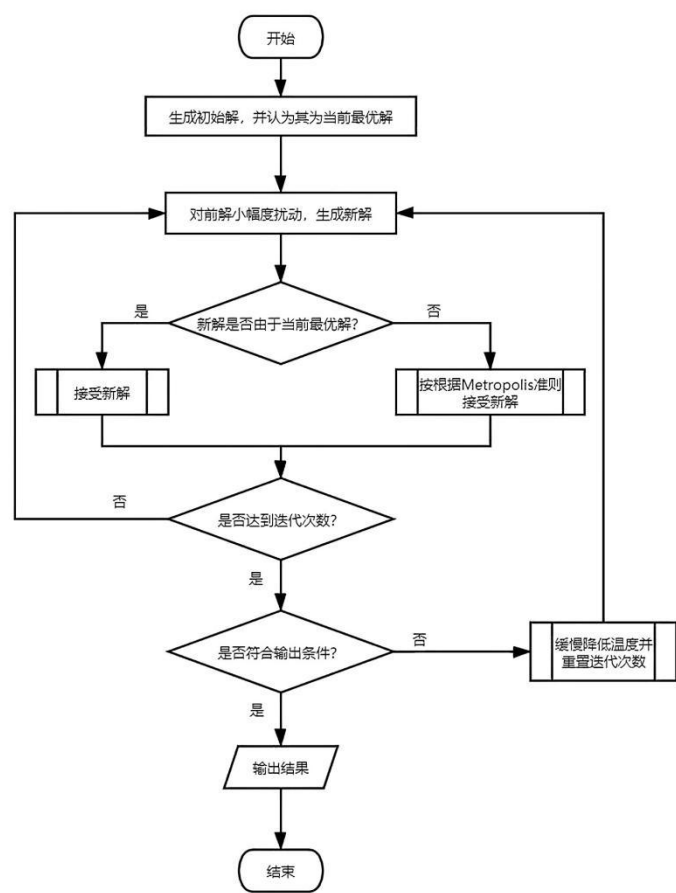


图 29 模拟退火算法流程图

图 30 展示了模拟退火算法在迭代过程中的目标函数值变化情况。从图中可以看出，随着迭代次数的增加，曲线逐渐下降，在 1100 次迭代后逐渐收敛到一个稳定的状态，表明算法在逐渐接近最优解，但还不算是一个好的结果，因此，我们建立多目标粒子群优化算法，与其对比，择优选取输出结果。

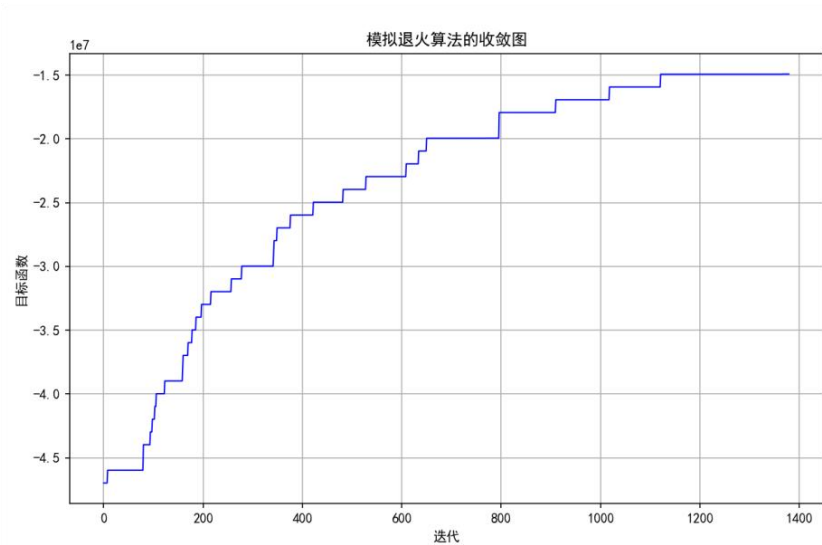


图 30 模拟退火算法收敛图

粒子群优化算法(Particle Swarm Optimization, PSO)是一种基于群体协作的随机搜索算法,通过模拟鸟群觅食行为而发展起来。该算法已被广泛应用于许多领域,包括神经网络训练、化工系统、图像处理、电力系统、生物信息、机械设计、医学等。在电商补货问题中,粒子群优化算法可以应用于优化补货策略,提高库存周转率和顾客满意度。粒子群算法流程如图 31 所示。

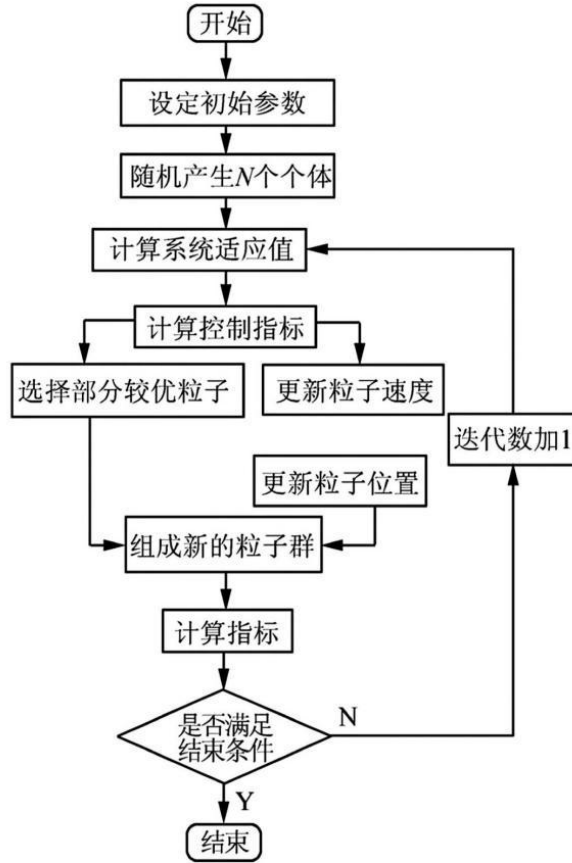


图 31 粒子群算法流程图

本题利用多目标粒子群算法的主要流程可以总结为以下几个步骤:

①初始化: 设置参数: 定义粒子群的大小 N , 最大迭代次数, 惯性权重 w , 加速常数 c_1 和 c_2 , 以及用于多目标优化的外部档案 (Pareto 前沿) 的大小。生成初始粒子, 随机生成 N 个粒子, 每个粒子代表一个品类分配方案, 确保每个品类最多分配给三个仓库, 并满足件型和高级品类的标签约束。初始化速度和位置, 为每个粒子随机初始化速度和位置向量。评估目标函数, 计算每个粒子的品类关联度 f_1 、产能利用率 f_2 、仓能利用率 f_3 、总仓租成本 f_4 (取负值以与最大化目标一致) 和品类分仓数 f_5 (同样取负值)。

②更新 Pareto 前沿: 将初始粒子集加入外部档案, 并根据 Pareto 支配关系进行排序。如果档案大小超过, 则移除支配或被支配的粒子, 直到档案大小满足要求。

③更新粒子的速度和位置: 根据适应度函数计算每个粒子的适应度值, 然后根据粒子的速度和位置更新公式, 更新每个粒子的速度和位置。

速度更新公式为:

$$v_{id}^{k+1} = \omega v_{id}^k + c_1 r_1 (p_{id}^k - x_{id}^k) + c_2 r_2 (p_{gd-1}^k - x_{ad}^k) \quad (7-15)$$

位置更新公式为:

$$z_{id}^{k+1} = z_{id}^k + v_{id}^{k+1} \quad (7-16)$$

其中 $i = 1, 2, \dots, m$ ，表示粒子个数； $d = 1, 2, \dots, D$ ，表示维数； K 表示当前进化代数，或者说是当前迭代次数； z_{id}^k 为粒子当前的位置； ω 为惯性因子，为非负值。值较大时，全局寻优能力强，局部寻优能力弱；值较小时，全局寻优能力弱，局部寻优能力强。

④更新个体极值和全局极值：对每个粒子，将其适应度与其经历过的最好位置进行比较，如果优于该位置，则将其作为该粒子的最好位置。同时，对每个粒子，将其适应度与群体经历过的最好位置进行比较，如果优于该位置，则将其作为当前群体的最好位置。

⑤终止条件：如果达到预设的终止条件（如迭代次数或误差阈值），则停止迭代，输出全局最优解。否则，返回第 2 步继续迭代。

计算公式为：

$$\omega(k) = \frac{k_{max} - k}{k_{max}}(\omega_{ini} - \omega_{end}) + \omega_{end} \quad (7-17)$$

其中， ω_{ini} 为惯性因子初始值，通常设置为 0.4； ω_{end} 为种群迭代到最大进化次数时的惯性因子值，通常设置为 0.9； ω_{max} 为最大迭代次数通过以上步骤。

如图 32 所示，根据粒子群优化算法的收敛图趋势，可以看出，在达到近 200 的迭代次数后，其目标函数值趋于稳定，即目标函数值不再有明显下降。

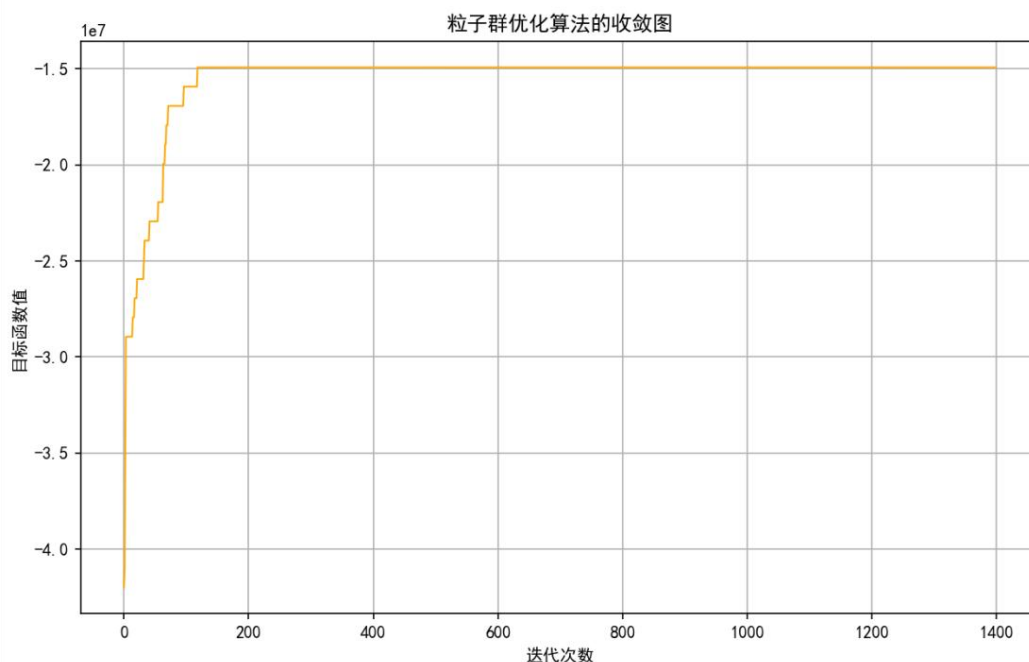


图 32 粒子群算法收敛图趋势

与模拟退火算法相比，粒子群优化算法更快找的最优解，因此，我们选择用粒子群优化算法输出“一品多仓”方案结果。通过上述求解，得到各商品的仓库分配结果见附件 1 中的 result 表格，其中问题 3 所需商品分仓结果如表 11 所示，根据题目，允许一品放多仓，但同一品类存放的仓库数量不能超过 3 个的条件，其空缺处表示，品类 1 的库存数量，可以完全放在 31 仓库就可以放下，无需再放入其它仓库。

表 11 “一品多仓”分仓方案

	warehouse	warehouse	warehouse
category1	warehouse31		
category31	warehouse110	warehouse131	
category61	warehouse25	warehouse112	
category91	warehouse9	warehouse10	warehouse52
category121	warehouse31	warehouse33	warehouse109
category151	warehouse36	warehouse114	warehouse132
category181	warehouse55		
category211	warehouse78	warehouse85	warehouse90
category241	warehouse44	warehouse54	
category271	warehouse117	warehouse119	warehouse130
category301	warehouse129	warehouse137	
category331	warehouse2	warehouse37	warehouse112

8 模型的评价和推广

8.1 模型的评价

8.1.1 模型的优点

(1) LSTM 神经网络模型的优点

LSTM 模型凭借其独特的门控机制（包括遗忘门、输入门和输出门）以及细胞状态，在多个方面展现出显著优势。它能够有效地捕捉序列数据中的长期依赖关系，这在处理时间序列数据、自然语言处理等领域尤为重要。LSTM 通过精细控制信息流动，在一定程度上缓解了梯度消失/爆炸问题，从而能够处理更长的序列数据。作为复杂的非线性单元，LSTM 可以集成到更大型的深度神经网络中，提升模型的表达能力和泛化能力。

(2) “一品一仓”方案的优点

在搜索过程中，分枝定界法通过剪枝策略，即放弃那些界限超出已知可行解的子集，从而避免了不必要的搜索，提高了算法的效率。分枝定界法特别擅长处理包含离散变量的优化问题，这是许多其他算法难以做到的。通过分枝定界法精确计算每个商品在特定仓库的存储和运输成本，可以优化总成本。

(3) “一品多仓”方案的优点

相比“一品一仓”，“一品多仓”方案允许一个品类在多个仓库中存储，这增加了库存的灵活性。然而，实施该方案时也需要考虑多个约束条件，如品类相似性和关联性、

库存和出库量比例的一致性等，以确保方案的可行性和有效性。同时，需要设计合理的评估机制来比较不同方案下各个业务指标的表现，并找到多个目标之间的最佳平衡点。在求解过程中采用多目标粒子群优化算法，该算法全局搜索能力强，能跳出局部最优解探索不同区域。该算法概念简单，易于实现，且本质并行，适合多处理器系统。此收敛性能好，能快速向最优解靠近。

8.1.2 模型的不足

(1) LSTM 神经网络模型的不足

LSTM 处理序列数据时只能顺序计算，限制了大规模数据的并行处理效率。其内部结构复杂，训练过程耗时且需更多计算资源。并且其复杂性导致难以解释其决策过程。

(2) “一品一仓”方案的不足

在解决“一品一仓”方案时，需要用分枝定界法，而其方法搜索过程中内存消耗大，需存储大量信息。大规模搜索空间下，分枝定界法计算复杂度高，运行时间长。并且其效率依赖界估计准确性，不准确可能导致效率低下。

(3) “一品多仓”方案的不足

在解决“一品多仓”方案时，需要用多目标粒子群优化算法，该算法在处理多目标优化时，计算复杂度高，尤其在目标函数多或问题规模大时负担显著增加。同时，该算法可能过于追求个别目标最优，难以找到均衡解。

8.2 模型的推广

8.2.1 科学性

LSTM 模型、分枝定界法和多目标粒子群优化算法在科学研究中均展现出高度的科学性和有效性。LSTM 模型在时间序列数据处理领域（如自然语言处理、语音识别、机器翻译等）得到了广泛应用和验证。分枝定界法基于数学优化理论，通过系统搜索解空间来找到最优解，其有效性得到了广泛认可。多目标粒子群优化算法则基于粒子群优化思想，擅长处理多目标优化问题，能够找到多个目标之间的最佳平衡解（Pareto 前沿），同样具有坚实的理论基础和广泛的实验验证。

8.2.2 拓展性

LSTM 模型能与其他深度学习模型结合，提升性能和适用范围。分枝定界法和多目标粒子群优化算法则可与其他优化算法融合，形成更高效的混合算法，加速求解并提高质量。此外，多目标粒子群优化算法的应用领域广泛，包括连续和离散优化等问题

参考文献

- [1] 陈悦,陈超美,刘则渊等.CiteSpace 知识图谱的方法论功能[J].科学学研究,2015,33(02): 242-253.
- [2] 李长春.大数据背景下的商品需求预测与分仓规划[J].数学的实践与认识,2017,47(07): 70-79.

- [3] 王之威.DJ 公司分仓物流网络优化研究[J].中国高新技术企业,2016,(20):23-25.
- [4] 刘顺,黄晓宇,鲜征征,等.基于超图嵌入表示的商品分仓模型[J/OL].中国管理科学,1-12[2024-10-31].2022.0102.
- [5] 谢新连,王余宽,许小卫,等.基于随机森林算法的港口集装箱吞吐量预测方法[J].重庆交通大学学报(自然科学版),2022,41(02):15-20.
- [6] 黄永福.高质量发展视角下广东省“十四五”物流需求量预测研究——基于 GM(1,N)灰色模型[J].经营与管理,2022,(10):174-179.
- [7] 潘思羽,张美玲.基于 BP 神经网络的甘肃省二氧化碳排放预测及影响因素研究[J].环境工程,2023,41(07):61-68+85.
- [8] 陈佳,刘冬雪,武大硕.基于特征选取与 LSTM 模型的股指预测方法研究[J].计算机工程与应用,2019,55(06):108-112.
- [9] 丁敬安.特征选择下的集成学习模型商品需求预测及分仓研究[D].安徽工业大学,2018.
- [10] 施政,杨辉,曹翰.会议分组的优化.数学的实践与认识.1998,27(4):335~347.

附录

1.支撑文件列表

问题 1-3 计算结果，见附件 1.

问题 1 支撑材料，见附件 2.

问题 2 支撑材料，见附件 3.

2.附录内容

附录一: 问题 1 的缺失值处理

程序编号	1	程序说明	问题 1：缺失值处理	编程软件	Python
<pre>import pandas as pd # 加载数据文件，指定编码为 'gbk'（适用于中文字符文件） file_path = 'D:/桌面 1/2024 妈妈杯/附件 1.csv' data = pd.read_csv(file_path, encoding='gbk') data df = data # 检查缺失值的个数 missing_values_count = df.isnull().sum() # 打印结果 print("缺失值个数:") print(missing_values_count) import pandas as pd # 加载数据文件，指定编码为 'gbk'（适用于中文字符文件） file_path = 'D:/桌面 1/2024 妈妈杯/附件 2.csv' data = pd.read_csv(file_path, encoding='gbk') data</pre>					

```
df = data

# 检查缺失值的个数
missing_values_count = df.isnull().sum()

# 打印结果
print("缺失值个数:")
print(missing_values_count)
```

附录二: 问题 1 的异常值处理

程序编号	2	程序说明	问题 1: 异常值处理	编程软件	Python
<pre>import pandas as pd # 加载数据文件, 指定编码为 'gbk' (适用于中文字符文件) file_path = 'D:/桌面 1/2024 妈妈杯/附件 1.csv' data = pd.read_csv(file_path, encoding='gbk') data df = data # 检查空值 null_values_count = df['品类'].isnull().sum() print(f'品类列空值个数: {null_values_count}') # 假设月份是数字, 并且我们想要使用 IQR 方法来检测异常值 import numpy as np # 将月份列转换为数值类型 (如果还不是的话) df['月份'] = pd.to_numeric(df['月份'], errors='coerce') # 计算 IQR Q1 = df['月份'].quantile(0.25) Q3 = df['月份'].quantile(0.75) IQR = Q3 - Q1 # 设定异常值的界限 (通常使用 1.5 倍 IQR) lower_bound = Q1 - 1.5 * IQR</pre>					

```

upper_bound = Q3 + 1.5 * IQR

# 检测异常值
outliers_count = ((df['月份'] < lower_bound) | (df['月份'] > upper_bound)).sum()
print(f'月份列异常值个数: {outliers_count}')

# 检查“库存量”列是否为负值，并计算负值的个数
negative_sales = df['库存量'] < 0
negative_sales_count = negative_sales.sum()

print(f'库存量负值（异常值）个数: {negative_sales_count}')
import pandas as pd
import numpy as np
# 加载数据文件，指定编码为 'gbk'（适用于中文字符文件）
file_path = 'D:/桌面 1/2024 妈妈杯/附件 2.csv'
data = pd.read_csv(file_path, encoding='gbk')
data
df = data
# 检测“品类”列的异常值
# 检查空值
null_values_category = df['品类'].isnull().sum()
print(f'品类列空值个数: {null_values_category}')

# 检测“日期”列的异常值
# 首先确保日期列是日期类型
try:
    df['日期'] = pd.to_datetime(df['日期'], errors='coerce')
except ValueError as e:
    print(f'日期列转换失败: {e}')
    # 处理转换失败的情况，这里简单跳过
# 检查空值
null_values_date = df['日期'].isnull().sum()
print(f'日期异常值个数: {null_values_date}')

# 检测“销量”列的异常值
# 检查“销量”列是否为负值，并计算负值的个数

```

```
negative_sales = df['销量'] < 0
negative_sales_count = negative_sales.sum()
print(f'销量列负值（异常值）个数: {negative_sales_count}')
```

附录三: 问题 1 的库存量预测

程序编号	3	程序说明	问题 1: 库存量预测	编程软件	Python
<pre>import pandas as pd import numpy as np from sklearn.ensemble import RandomForestRegressor from sklearn.metrics import mean_absolute_percentage_error from sklearn.tree import DecisionTreeRegressor from xgboost import XGBRegressor from keras.models import Sequential from keras.layers import Dense from keras.optimizers import Adam from sklearn.preprocessing import MinMaxScaler import matplotlib.pyplot as plt import matplotlib.dates as mdates import seaborn as sns import warnings warnings.filterwarnings('ignore') plt.rcParams['font.sans-serif']=['SimHei'] plt.rcParams['axes.unicode_minus']=False # 加载数据文件，指定编码为 'gbk'（适用于中文字符文件） file_path = 'D:/桌面 1/2024 妈妈杯/附件 1.csv' data = pd.read_csv(file_path, encoding='gbk') data1 = data[data['品类'] == 'category1'] data1 = data1.sort_values(by='月份', ascending=True).reset_index(drop=True) data1['月份'] = pd.to_datetime(data1['月份'], errors='coerce')</pre>					

```

# 去除可能的缺失值
data1 = data1.dropna(subset=['月份'])

# 设定特征和目标变量
X = data1[['月份']]
y = data1['库存量']

train_data = data1.iloc[:-3]
test_data = data1.iloc[-3:]

train_series = train_data['库存量'].values
test_series = test_data['库存量'].values

# 使用随机森林模型进行训练
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(train_data[['月份']], train_series)

# 对测试集进行预测
test_pred = model.predict(test_data[['月份']])

# 计算 MAPE
RandomForest_mape = mean_absolute_percentage_error(test_series, test_pred)
print("随机森林模型的 MAPE:", RandomForest_mape)

all_dates = pd.concat([train_data['月份'], test_data['月份']]).reset_index(drop=True)
all_inventory = np.concatenate([train_series, test_pred])
true_inventory = np.concatenate([train_series, test_series])

# 绘制预测结果与实际值的对比图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-.')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试集真实值', color='green', linestyle='-.')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_pred, label='随机森林模型预测值', color='red', linestyle='-.')

```

```

plt.xlabel('时间')
plt.ylabel('库存量')
plt.title('随机森林模型 - 真实值与预测值对比')
plt.legend()
plt.show()

# 决策树回归模型预测与 MAPE 计算
def create_sliding_window_dataset(series, step=2):
    X, y = [], []
    for i in range(len(series) - step):
        X.append(series[i:i + step])
        y.append(series[i + step])
    return np.array(X), np.array(y)

# 创建滑窗数据集，使用步阶为 2
X_train, y_train = create_sliding_window_dataset(train_series, step=1)
X_test, y_test = create_sliding_window_dataset(np.concatenate([train_series[-1:], test_series]),
step=1)

# 构建决策树回归模型
tree_model = DecisionTreeRegressor(random_state=0)
tree_model.fit(X_train, y_train)

# 进行预测并计算 MAPE
tree_forecast = tree_model.predict(X_test)
tree_mape = mean_absolute_percentage_error(y_test, tree_forecast)
tree_mape

# 决策树回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试集真实值')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), tree_forecast, label='决

```

```

策树预测值')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("决策树回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# XGBoost 回归模型预测与 MAPE 计算
xgb_model = XGBRegressor(objective='reg:squarederror', n_estimators=10, max_depth=3,
random_state=0)
xgb_model.fit(X_train, y_train)
xgb_forecast = xgb_model.predict(X_test)
xgb_mape = mean_absolute_percentage_error(test_series, xgb_forecast)
xgb_mape

# XGBoost 回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), xgb_forecast,
label='XGBoost 预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("XGBoost 回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

#灰色预测（GM(1,1)）模型预测与 MAPE 计算
def grey_model_forecast(series, forecast_steps=3):
    n = len(series)
    X1 = np.cumsum(series)
    Z1 = (X1[:-1] + X1[1:]) / 2.0
    B = np.vstack([-Z1, np.ones(n - 1)]).T
    Y = series[1:]

```

```

result = np.linalg.lstsq(B, Y, rcond=None)[0]
a, b = result
def grey_forecast(k):
    return (series[0] - b / a) * np.exp(-a * k) + b / a
forecast = [grey_forecast(i) for i in range(n, n + forecast_steps)]
return np.array(forecast)
grey_forecast = grey_model_forecast(train_series, forecast_steps=len(test_series))
grey_mape = mean_absolute_percentage_error(test_series, grey_forecast)
grey_mape

# 灰色预测模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), grey_forecast, label='灰
色预测模型预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("灰色预测模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# BP 神经网络
data1['月份'] = pd.to_datetime(data1['月份'], errors='coerce')
data1['年份'] = data1['月份'].dt.year
data1['月份_num'] = data1['月份'].dt.month
# 设定特征和目标变量
X = data1[['年份', '月份_num']].values
y = data1['库存量'].values

# 数据归一化处理
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

```

```

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.reshape(-1, 1)).flatten()

# 将数据分为训练集和测试集（最后 3 个数据作为测试集）
train_size = len(X_scaled) - 3
train_X, test_X = X_scaled[:train_size], X_scaled[train_size:]
train_y, test_y = y_scaled[:train_size], y_scaled[train_size:]

train_series_original = data1['库存量'].iloc[:train_size]
test_series_original = data1['库存量'].iloc[train_size:]

# 构建 BP 神经网络模型
model = Sequential()
model.add(Dense(64, input_dim=2, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='linear'))

# 编译模型
model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# 训练模型
model.fit(train_X, train_y, epochs=100, batch_size=10, verbose=1)

# 预测
predictions = model.predict(test_X)
predictions_scaled_back = scaler_y.inverse_transform(predictions) # 逆归一化预测值
test_y_scaled_back = scaler_y.inverse_transform(test_y.reshape(-1, 1)).flatten() # 逆归一化测试集真实值

# 计算 MAPE
BPNN_mape = mean_absolute_percentage_error(test_y_scaled_back, predictions_scaled_back)
print("BP 神经网络模型的 MAPE:", BPNN_mape)

plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_y)), scaler_y.inverse_transform(train_y.reshape(-1, 1)).flatten(),

```

```

label='训练集真实值', color='orange', linestyle='-') # 使用逆归一化后的训练集真实值
plt.plot(np.arange(len(train_y), len(train_y) + len(test_y)), test_y_scaled_back, label='测试集真实值', color='green', linestyle='--') # 使用逆归一化后的测试集真实值
plt.plot(np.arange(len(train_y), len(train_y) + len(test_y)), predictions_scaled_back, label='BP 神经网络预测值', color='red', linestyle='-.') # 使用逆归一化后的预测值
plt.xlabel('时间')
plt.ylabel('库存量')
plt.title('BP 神经网络 - 真实值与预测值对比')
plt.legend()
plt.show()

import time

start_time = time.time()

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator
X_train_lstm = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test_lstm = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))
a = [i for i in range(1000)]
b = [i**2 for i in a]
end_time = time.time()

print("执行时间: {:.6f} 秒".format(end_time - start_time))

# 构建 LSTM 模型
lstm_model = Sequential()
lstm_model.add(LSTM(50, activation='relu', input_shape=(X_train_lstm.shape[1], 1)))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer=Adam(learning_rate=0.01), loss='mse')

# 训练 LSTM 模型
lstm_model.fit(X_train_lstm, y_train, epochs=120, verbose=0)

```

```

# 进行预测并计算 MAPE
lstm_forecast = lstm_model.predict(X_test_lstm, verbose=0).flatten()
lstm_mape = mean_absolute_percentage_error(y_test, lstm_forecast)

lstm_mape

# LSTM 回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), lstm_forecast,
label='LSTM 预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("LSTM 回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# 汇总 MAPE 值并绘图
mape_values = {
    '随机森林': RandomForest_mape,
    'XGBoost': xgb_mape,
    '灰色预测': grey_mape,
    'BP 神经网络': BPNN_mape,
    'LSTM': lstm_mape
}

# 定义一个颜色列表，为每个模型分配不同的颜色
colors = ['skyblue', 'gold', 'lightgreen', 'lightcoral', 'lightsteelblue']

plt.figure(figsize=(8, 5))
# 使用颜色列表为每个条形设置不同的颜色
plt.bar(mape_values.keys(), mape_values.values(), color=colors, edgecolor='black', linewidth=1)

```

```
plt.xlabel("模型")
plt.ylabel("MAPE")
plt.title("不同模型的 MAPE 比较")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

附录四：问题 1 的销量预测

程序编号	4	程序说明	问题 1：销量预测	编程软件	Python
<pre>import pandas as pd import numpy as np from sklearn.ensemble import RandomForestRegressor from statsmodels.tsa.arima.model import ARIMA from scipy.optimize import minimize from sklearn.tree import DecisionTreeRegressor from xgboost import XGBRegressor from sklearn.metrics import mean_absolute_percentage_error from keras.models import Sequential from keras.layers import Dense from keras.optimizers import Adam from sklearn.preprocessing import MinMaxScaler import matplotlib.pyplot as plt import matplotlib.dates as mdates import seaborn as sns import warnings warnings.filterwarnings('ignore') plt.rcParams['font.sans-serif']=['SimHei'] plt.rcParams['axes.unicode_minus']=False # 加载数据文件，指定编码为 'gbk' file_path = 'D:/桌面 1/2024 妈妈杯/附件 2.csv' data = pd.read_csv(file_path, encoding='gbk') data</pre>					

```

data['日期'] = pd.to_datetime(data['日期'], errors='coerce')

# 去除可能的缺失值
data = data.dropna(subset=['日期'])
data = data.sort_values(by='日期', ascending=False).reset_index(drop=True)
data1 = data[data['品类'] == 'category1']
data1.reset_index(inplace=True, drop=True)
n = 7
data_modified = data1.copy()
data_modified['迭代数'] = [(i % n) + 1 for i in range(len(data_modified))]
data_modified['周期'] = [i // n for i in range(len(data_modified))]
data_modified
last_cycle = data_modified['周期'].max()
last_cycle_count = data_modified[data_modified['周期'] == last_cycle].shape[0]

if last_cycle_count != n:
    data_modified = data_modified[data_modified['周期'] != last_cycle]
data_modified

data_modified[data_modified['周期']==25]
tt=[]
for d in data_modified['周期'].unique():
    sumv=data_modified[data_modified['周期']==d]['销量'].sum()
    tt.append((data_modified[data_modified['周期']==d]['销量']/sumv).to_list())
tt=pd.DataFrame(tt).mean()
tt
data_modified.groupby(['周期'])['销量'].sum()
data1=pd.DataFrame(data_modified.groupby(['周期'])['销量'].sum())
data1.reset_index(inplace=True)
data1 = data1.sort_values(by='周期', ascending=False).reset_index(drop=True)
data1.reset_index(inplace=True)
data1
from datetime import datetime
train_data = data1.iloc[:-7]
test_data = data1.iloc[-7:]

```

```

train_series = train_data['销量'].values
test_series = test_data['销量'].values

from statsmodels.tsa.stattools import adfuller
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

# 灰色预测 (GM(1,1)) 模型预测与 MAPE 计算
def grey_model_forecast(series, forecast_steps=3):
    n = len(series)
    X1 = np.cumsum(series)
    Z1 = (X1[:-1] + X1[1:]) / 2.0
    B = np.vstack([-Z1, np.ones(n - 1)]).T
    Y = series[1:]
    result = np.linalg.lstsq(B, Y, rcond=None)[0]
    a, b = result
    def grey_forecast(k):
        return (series[0] - b / a) * np.exp(-a * k) + b / a
    forecast = [grey_forecast(i) for i in range(n, n + forecast_steps)]
    return np.array(forecast)

grey_forecast = grey_model_forecast(train_series, forecast_steps=len(test_series))
grey_mape = mean_absolute_percentage_error(test_series, grey_forecast)
grey_mape

# 灰色预测模型绘图
plt.figure(figsize=(10, 6))
plt.bar(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.bar(np.arange(len(train_series), len(train_series) + len(test_series)), grey_forecast, label='灰色预测模型预测值', color='red', linestyle='-')
plt.bar(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试集真实值', color='green', linestyle='--')
plt.xlabel("时间")
plt.ylabel("库存量")

```

```

plt.title("灰色预测模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# 决策树回归模型预测与 MAPE 计算
# 生成滑窗数据集的函数
def create_sliding_window_dataset(series, step=2):
    X, y = [], []
    for i in range(len(series) - step):
        X.append(series[i:i + step])
        y.append(series[i + step])
    return np.array(X), np.array(y)

# 创建滑窗数据集，使用步阶为 2
X_train, y_train = create_sliding_window_dataset(train_series, step=1)
X_test, y_test = create_sliding_window_dataset(np.concatenate([train_series[-1:], test_series]),
step=1)

# 构建决策树回归模型
tree_model = DecisionTreeRegressor(random_state=0)
tree_model.fit(X_train, y_train)

# 进行预测并计算 MAPE
tree_forecast = tree_model.predict(X_test)
tree_mape = mean_absolute_percentage_error(y_test, tree_forecast)
tree_mape

# 决策树回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), tree_forecast, label='决
策树预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")

```

```

plt.title("决策树回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# XGBoost 回归模型预测与 MAPE 计算
xgb_model = XGBRegressor(objective='reg:squarederror', n_estimators=10, max_depth=3,
random_state=0)
xgb_model.fit(X_train, y_train)
xgb_forecast = xgb_model.predict(X_test)
xgb_mape = mean_absolute_percentage_error(test_series, xgb_forecast)
xgb_mape

# XGBoost 回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), xgb_forecast,
label='XGBoost 预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("XGBoost 回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

import time
start_time = time.time()
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.sequence import TimeseriesGenerator

X_train_lstm = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test_lstm = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

# 构建 LSTM 模型

```

```

lstm_model = Sequential()
lstm_model.add(LSTM(50, activation='relu', input_shape=(X_train_lstm.shape[1], 1)))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer=Adam(learning_rate=0.01), loss='mse')

# 训练 LSTM 模型
lstm_model.fit(X_train_lstm, y_train, epochs=120, verbose=0)

# 进行预测并计算 MAPE
lstm_forecast = lstm_model.predict(X_test_lstm, verbose=0).flatten()
lstm_mape = mean_absolute_percentage_error(y_test, lstm_forecast)
lstm_mape

# LSTM 回归模型绘图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), lstm_forecast,
label='LSTM 预测值', color='red', linestyle='-.')
plt.xlabel("时间")
plt.ylabel("库存量")
plt.title("LSTM 回归模型 - 真实值与预测值对比")
plt.legend()
plt.show()

# 使用随机森林模型进行训练
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(train_data[['周期']], train_series)

# 对测试集进行预测
test_pred = model.predict(test_data[['周期']])

# 计算 MAPE
RandomForest_mape = mean_absolute_percentage_error(test_series, test_pred)
print("随机森林模型的 MAPE:", RandomForest_mape)

```

```

all_dates = pd.concat([train_data['周期'], test_data['周期']]).reset_index(drop=True)
all_inventory = np.concatenate([train_series, test_pred])
true_inventory = np.concatenate([train_series, test_series])

# 绘制预测结果与实际值的对比图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_series)), train_series, label='训练集真实值', color='orange',
linestyle='-')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_series, label='测试
集真实值', color='green', linestyle='--')
plt.plot(np.arange(len(train_series), len(train_series) + len(test_series)), test_pred, label='随机森
林模型预测值', color='red', linestyle='-.')
plt.xlabel('时间')
plt.ylabel('库存量')
plt.title('随机森林模型 - 真实值与预测值对比')
plt.legend()
plt.show()

# BP 神经网络
def create_lag_features(data, lag=1):
    lags = [data['销量'].shift(i) for i in range(1, lag+1)]
    lags.append(data['周期'])
    return pd.concat(lags, axis=1).dropna()

lag_features = create_lag_features(data1, lag=3)
lag_features.columns = [f'lag_{i}' for i in range(1, 4)] + ['周期']

train_X = data1.iloc[:-7]
test_X = data1.iloc[-7:]

train_y = train_data['销量'].values
test_y = test_data['销量'].values

# 数据归一化处理
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

```

```

train_X_scaled = scaler_X.fit_transform(train_X)
test_X_scaled = scaler_X.transform(test_X)
train_y_scaled = scaler_y.fit_transform(train_y.reshape(-1, 1)).flatten()
test_y_scaled = scaler_y.transform(test_y.reshape(-1, 1)).flatten()

# 构建 BP 神经网络模型
model = Sequential()
model.add(Dense(64, input_dim=train_X_scaled.shape[1], activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='linear'))

# 编译模型
model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')

# 训练模型
model.fit(train_X_scaled, train_y_scaled, epochs=100, batch_size=10, verbose=1)

# 预测
predictions_scaled = model.predict(test_X_scaled)
predictions = scaler_y.inverse_transform(predictions_scaled)
test_y_original = scaler_y.inverse_transform(test_y_scaled.reshape(-1, 1)).flatten()

# 计算 MAPE
BPNN_mape = mean_absolute_percentage_error(test_y_original, predictions)
print("BP 神经网络模型的 MAPE:", BPNN_mape)

# 绘制对比图
plt.figure(figsize=(10, 6))
plt.plot(np.arange(len(train_y)), scaler_y.inverse_transform(train_y.reshape(-1, 1)).flatten(),
label='训练集真实值', color='orange', linestyle='-')
plt.plot(np.arange(len(train_y), len(train_y) + len(test_y)), test_y_original, label='测试集真实值',
color='green', linestyle='--')
plt.plot(np.arange(len(train_y), len(train_y) + len(test_y)), predictions, label='BP 神经网络预测值',
color='red', linestyle='-.')
plt.xlabel('时间')
plt.ylabel('库存量')

```

```

plt.title('BP 神经网络 - 真实值与预测值对比')
plt.legend()
plt.show()

# 汇总 MAPE 值并绘图
mape_values = {
    '随机森林': RandomForest_mape,
    'XGBoost': xgb_mape,
    '灰色预测': grey_mape,
    'BP 神经网络': BPNN_mape,
    'LSTM': lstm_mape }
# 定义一个颜色列表，为每个模型分配不同的颜色
colors = ['skyblue', 'gold', 'lightgreen', 'lightcoral', 'lightsteelblue']

plt.figure(figsize=(8, 5))
# 使用颜色列表为每个条形设置不同的颜色
plt.bar(mape_values.keys(), mape_values.values(), color=colors, edgecolor='black', linewidth=1)
plt.xlabel("模型")
plt.ylabel("MAPE")
plt.title("不同模型的 MAPE 比较")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

附录五: 问题 2 的“一品一仓分仓方案”

程序编号	5	程序说明	问题 2: “一品一仓分仓方案”	编程软件	Python
<pre> # 建立模型 IPr = Model('IPr') # 建立决策变量 X = [[[] for _ in range(0, data.vehicleNum)] for _ in range(0, data.nodeNum)] for _ in range(0, data.nodeNum)] S = [[[] for _ in range(0, data.vehicleNum)] for _ in range(0, data.nodeNum)] for i in range(0,data.nodeNum): for j in range(0,data.nodeNum): </pre>					

```

        if i != j:
            for k in range(0,data.vehicleNum):
                X[i][j][k] = IPr.addVar(vtype=GRB.CONTINUOUS, lb=0, ub=1,
name=f'X_{i}_{j}_{k}')
for i in range(0, data.nodeNum):
    for k in range(0, data.vehicleNum):
        S[i][k] = IPr.addVar(vtype=GRB.CONTINUOUS, lb=0, name=f'S_{i}_{k}')
IPr.update()
# 建立目标函数
obj = LinExpr(0)
for i in range(0,data.nodeNum):
    for j in range(0,data.nodeNum):
        if i != j:
            for k in range(0,data.vehicleNum):
                obj.addTerms(data.distanceMatrix[i][j], X[i][j][k])
IPr.setObjective(obj, sense=GRB.MINIMIZE)
# 建立约束条件 (2)
cnt = 0
for i in range(1, data.nodeNum-1):
    expr = LinExpr(0)
    for j in range(0, data.nodeNum):
        if i != j:
            for k in range(0, data.vehicleNum):
                expr.addTerms(1, X[i][j][k])
    cnt += 1
    IPr.addConstr(expr == 1, f'C1_{cnt}')
# 建立约束条件 (3)
cnt = 0
for k in range(0, data.vehicleNum):
    expr = LinExpr(0)
    for j in range(1,data.nodeNum):
        expr.addTerms(1,X[0][j][k])
    cnt += 1
    IPr.addConstr(expr == 1, f'C2_{cnt}')
# 建立约束条件 (4)
cnt = 0

```

```

for k in range(0, data.vehicleNum):
    expr = LinExpr(0)
    for i in range(0,data.nodeNum-1):
        expr.addTerms(1,X[i][data.nodeNum-1][k])
    cnt += 1
    IPr.addConstr(expr == 1, f'C3_{cnt}')
# 建立约束条件 (5)
cnt = 0
for k in range(0,data.vehicleNum):
    for h in range(1, data.nodeNum-1):
        expr = LinExpr(0)
        for i in range(0,data.nodeNum):
            if i != h:
                expr.addTerms(1,X[i][h][k])
        for j in range(0,data.nodeNum):
            if j != h:
                expr.addTerms(-1, X[h][j][k])
        cnt += 1
        IPr.addConstr(expr == 0, f'C4_{cnt}')
# 建立约束条件 (6)
cnt = 0
for k in range(0,data.vehicleNum):
    for i in range(0,data.nodeNum):
        for j in range(0,data.nodeNum):
            if i != j:
                expr = LinExpr(0)
                expr.addTerms(1, S[i][k])
                expr.addTerms(M, X[i][j][k])
                expr.addTerms(-1, S[j][k])
                cnt += 1
                IPr.addConstr(expr + data.distanceMatrix[i][j] <= M, f'C5_{cnt}')
# 建立约束条件 (7)
cnt = 0
for i in range(1,data.nodeNum-1):
    for k in range(0,data.vehicleNum):
        expr = LinExpr(0)

```

```

        expr.addTerms(1,S[i][k])
        cnt += 1
        IPr.addConstr(expr <= data.dueTime[i], fC6_{cnt}_1')
        IPr.addConstr(expr >= data.readyTime[i], fC6_{cnt}_2')
# 建立约束条件 (8)
cnt = 0
for k in range(0,data.vehicleNum):
    expr = LinExpr(0)
    for i in range(1,data.nodeNum-1):
        for j in range(0,data.nodeNum):
            if i !=j:
                expr.addTerms(data.demand[i],X[i][j][k])
        cnt += 1
        IPr.addConstr(expr <= data.capacity, fC7_{cnt}'))
IPr.write('IPr_CVRPTW.lp')

```

附录六: 问题 3 的 “一品多仓分仓方案”

程序编号	6	程序说明	问题 3: “一品多仓分仓方案”	编程软件	Python
<pre> import pandas as pd import numpy as np import random import math import matplotlib.pyplot as plt import seaborn as sns plt.rcParams['font.sans-serif']=['SimHei'] # 中文支持 plt.rcParams['axes.unicode_minus']=False # 加载数据文件，指定编码为 'gbk'（适用于中文字符文件） file_path = 'D:/桌面 1/2024 妈妈杯/result.xlsx' inventory_forecast = pd.read_excel(file_path, sheet_name='月库存预测结果') sales_forecast = pd.read_excel(file_path, sheet_name='日销量预测结果') warehouse_data = pd.read_csv('D:/桌面 1/2024 妈妈杯/附件/附件 3.csv', encoding='gbk') </pre>					

```

association_data = pd.read_csv('D:/桌面 1/2024 妈妈杯/附件/附件 4.csv', encoding='gbk')
category_info = pd.read_csv('D:/桌面 1/2024 妈妈杯/附件/附件 5.csv', encoding='gbk')

# 数据预处理
sales_forecast.columns = ['品类'] + ['day{}'.format(i) for i in range(1, 93)]
inventory_forecast['I_i'] = inventory_forecast[['7 月预测值', '8 月预测值', '9 月预测值']].max(axis=1)
sales_forecast['S_i'] = sales_forecast.drop(columns=['品类']).sum(axis=1) / 93

# 创建关联度矩阵 a_{ik}
a_ik = {}
for idx, row in association_data.iterrows():
    i, k = row['品类 1'], row['品类 2']
    a_ik[(i, k)] = row['关联度']
    a_ik[(k, i)] = row['关联度']

categories = ['category{}'.format(i) for i in range(1, 351)]
warehouses = ['warehouse{}'.format(i) for i in range(1, 141)]

# 参数提取
I_i_dict = inventory_forecast.set_index('品类')['I_i'].to_dict()
S_i_dict = sales_forecast.set_index('品类')['S_i'].to_dict()
C_j_dict = warehouse_data.set_index('仓库')['仓容上限'].to_dict()
P_j_dict = warehouse_data.set_index('仓库')['产能上限'].to_dict()
R_j_dict = warehouse_data.set_index('仓库')['仓租日成本'].to_dict()

# 合并品类数据
category_data = pd.merge(pd.merge(inventory_forecast, sales_forecast, on='品类'),
category_info, on='品类')

delta_p = {}
delta_h = {}
for i in categories:
    for k in categories:
        if i != k:
            delta_p[(i, k)] = int(category_data.loc[category_data['品类'] == i, '件型

```

```

'].values[0] == category_data.loc[category_data['品类'] == k, '件型'].values[0])
        delta_h[(i, k)] = int(category_data.loc[category_data['品类'] == i, '高级品类
'].values[0] == category_data.loc[category_data['品类'] == k, '高级品类'].values[0])

lambda_p = 10
lambda_h = 5

def create_individual():
    individual = []
    for _ in categories:
        warehouse = random.choice(range(len(warehouses)))
        individual.append(warehouse)
    return individual

# 计算适应度
def fitness(individual):
    total_association = 0
    warehouse_inventory = {j: 0 for j in range(len(warehouses))}
    warehouse_capacity = {j: 0 for j in range(len(warehouses))}

    warehouse_categories = {j: [] for j in range(len(warehouses))}
    for i, w in enumerate(individual):
        warehouse_categories[w].append(i)
        warehouse_inventory[w] += I_i_dict[categories[i]]
        warehouse_capacity[w] += S_i_dict[categories[i]]

    penalty = 0
    for j in range(len(warehouses)):
        if warehouse_inventory[j] > C_j_dict[warehouses[j]]:
            penalty += 1e6
        if warehouse_capacity[j] > P_j_dict[warehouses[j]]:
            penalty += 1e6

    for j in range(len(warehouses)):
        categories_in_warehouse = warehouse_categories[j]
        num_categories = len(categories_in_warehouse)

```

```

        if num_categories > 1:
            for idx1 in range(num_categories):
                i = categories_in_warehouse[idx1]
                for idx2 in range(idx1 + 1, num_categories):
                    k = categories_in_warehouse[idx2]
                    key = (categories[i], categories[k])
                    a = a_ik.get(key, 0)
                    if a > 0 or delta_p.get(key, 0) == 1 or delta_h.get(key, 0) == 1:
                        delta_p_value = delta_p.get(key, 0)
                        delta_h_value = delta_h.get(key, 0)
                        A_ik = a + lambda_p * delta_p_value + lambda_h * delta_h_value
                        total_association += A_ik

    return total_association - penalty

# 粒子群优化算法
class Particle:
    def __init__(self, num_categories):
        self.position = [random.randint(0, len(warehouses) - 1) for _ in
range(num_categories)]
        self.velocity = [0] * num_categories
        self.best_position = self.position[:]
        self.best_value = float('-inf')

def pso(num_particles, num_iterations, w, c1, c2):
    particles = [Particle(len(categories)) for _ in range(num_particles)]
    global_best_position = None
    global_best_value = float('-inf')
    fitness_history = []

    for iteration in range(num_iterations):
        for particle in particles:
            # 计算当前粒子的适应度
            current_value = fitness(particle.position)

            # 更新粒子的个人最佳位置

```

```

        if current_value > particle.best_value:
            particle.best_value = current_value
            particle.best_position = particle.position[:]

        # 更新全局最佳位置
        if current_value > global_best_value:
            global_best_value = current_value
            global_best_position = particle.position[:]

        # 记录当前迭代的最佳适应度
        fitness_history.append(global_best_value)

        # 更新粒子的速度和位置
        for particle in particles:
            for i in range(len(categories)):
                r1, r2 = random.random(), random.random()
                cognitive_component = c1 * r1 * (particle.best_position[i] -
particle.position[i])
                social_component = c2 * r2 * (global_best_position[i] - particle.position[i])
                particle.velocity[i] = w * particle.velocity[i] + cognitive_component +
social_component
                particle.position[i] = int(np.clip(particle.position[i] + particle.velocity[i], 0,
len(warehouses) - 1))

        return global_best_position, global_best_value, fitness_history

# 参数设置
num_particles = 30
num_iterations = 1400
w = 0.5 # 惯性权重
c1 = 1.5 # 认知学习因子
c2 = 1.5 # 社会学习因子

# 运行粒子群优化算法
best_solution, best_energy, fitness_history = pso(num_particles, num_iterations, w, c1, c2)

```

```

# 绘制收敛图
plt.figure(figsize=(10, 6))
plt.plot(fitness_history, color='orange', linestyle='-', linewidth=1)
plt.xlabel('迭代次数')
plt.ylabel('目标函数值')
plt.title('粒子群优化算法的收敛图')
plt.grid(True)
plt.show()

# 输出最优解
best_individual = population[0]
assignment = []
for idx, warehouses_assigned in enumerate(best_individual):
    assigned_warehouses = [warehouses[w] for w in warehouses_assigned if w != -1]
    assignment.append({'品类': categories[idx], 'warehouse1': assigned_warehouses[0] if
len(assigned_warehouses) > 0 else "",
                        'warehouse2': assigned_warehouses[1] if
len(assigned_warehouses) > 1 else "",
                        'warehouse3': assigned_warehouses[2] if
len(assigned_warehouses) > 2 else ""})

assignment_df = pd.DataFrame(assignment)
assignment_df

assignment_df.to_excel('一品多仓分仓方案.xlsx', index=None)

```